

MEDICLINIC - PLATFORMĂ DE MANAGEMENT AL CLINICII MEDICALE

Coordinator științific:
Prof. dr. Ciprian-Bogdan Chiri

Candidat: Dawood Ahmed

Sesiunea: Iulie 2026

Contents

Cuprins	4
1 Tabel de figuri	6
1. Sinteza lucrării	7
2. Introducere	8
2.1 Scopul	8
2.2 Motivație	9
2.3 Importanța proiectului	9
2.4 Structura proiectului	10
3. Tehnologii web folosite	11
3.1 Limbajul de programare Java	11
3.2 Framework-ul Spring Boot	12
3.3 Spring Security.....	12
3.4 Spring Data JPA și Hibernate	13
3.5 MySQL	13
3.6 Thymeleaf.....	14
3.7 Bootstrap 5 și Chart.js	14
3.8 WebSocket și STOMP	15
3.9 Stripe API	15
3.10 DeepSeek AI	16
3.11 OpenPDF.....	16
3.12 Twilio SMS API	17
3.13 Docker și Docker Compose	17
3.14 JUnit 5 și Mockito	18
3.15 Maven	18
3.16 SpringDoc OpenAPI	19
4. Cerințele și specificațiile proiectului	19
4.1 Cerințe funcționale	19
4.1.1 Cerințe pentru rolul de pacient	19
4.1.2 Cerințe pentru rolul de doctor	20
4.1.3 Cerințe pentru rolul de administrator	21
4.2 Cerințe non-funcționale	21
4.2.1 Cerințe de securitate	21
4.2.2 Cerințe de performanță	22

4.2.3 Cerințe de disponibilitate	22
4.2.4 Cerințe de mentenabilitate	22
4.3 Restricții tehnice	22
5. Proiectarea	23
5.1 Schema bloc a aplicației	23
5.2 Diagrama use-case	24
5.3 Diagrama de clase	26
5.4 Diagrama de secvență pentru fluxul de programare.....	28
5.5 Diagrama de stare pentru Appointment	29
5.6 Schema bazei de date	31
6. Implementare	32
6.1 Structura pachetelor	32
6.2 Layerul de prezentare	32
6.2.1 PatientController	32
6.2.2 DoctorController	33
6.2.3 AdminController	34
6.2.4 MessagingController	34
6.2.5 PaymentController și StripeController	34
6.2.6 ClinicApiController	35
6.3 Layerul de business	35
6.3.1 PatientService	35
6.3.2 DoctorService	36
6.3.3 AiService	36
6.3.4 StripeService	37
6.3.5 NotificationService	37
6.3.6 SmartQueueService	38
6.4 Layerul de acces la date	38
6.5 Securitatea aplicației	39
6.6 Notificările în timp real	40
6.7 Integrările externe	41
1.1.1 Verificarea semnăturii webhook Stripe	41
1.1.2 Analiza simptomelor cu fallback	41
1.1.3 Conversia numerelor de telefon la format E.164	42
7. Testare și instalare	42
7.1 Testarea automată	42

7.2 Testarea manuală	43
7.3 Instalarea locală	44
7.4 Instalarea cu Docker	45
7.5 Deploy în Google Cloud Run	45
8. Concluzii și implementări ulterioare	47
9. Bibliografie	49

Cuprins

1. Sinteza lucrării	5
2. Introducere	6
2.1 Scopul	6
2.2 Motivație	7
2.3 Importanța proiectului	7
2.4 Structura proiectului	9
3. Tehnologii web folosite	10
3.1 Limbajul de programare Java	10
3.2 Framework-ul Spring Boot	11
3.3 Spring Security	12
3.4 Spring Data JPA și Hibernate	13
3.5 MySQL	14
3.6 Thymeleaf	15
3.7 Bootstrap 5 și Chart.js	16
3.8 WebSocket și STOMP	16
3.9 Stripe API	17
3.10 DeepSeek AI	18

3.11 OpenPDF	19
3.12 Twilio SMS API	19
3.13 Docker și Docker Compose	20
3.14 JUnit 5 și Mockito	20
3.15 Maven	21
3.16 SpringDoc OpenAPI	21
4. Cerințele și specificațiile proiectului	22
4.1 Cerințe funcționale	22
4.1.1 Cerințe pentru rolul de pacient	22
4.1.2 Cerințe pentru rolul de doctor	23
4.1.3 Cerințe pentru rolul de administrator	23
4.2 Cerințe non-funcționale	24
4.2.1 Cerințe de securitate	24
4.2.2 Cerințe de performanță	25
4.2.3 Cerințe de disponibilitate	25
4.2.4 Cerințe de mentenabilitate	25
4.3 Restricții tehnice	26
5. Proiectarea	27
5.1 Schema bloc a aplicației	27
5.2 Diagrama use-case	28
5.3 Diagrama de clase	30
5.4 Diagrama de secvență pentru fluxul de programare	32
5.5 Diagrama de stare pentru Appointment	34
5.6 Schema bazei de date	35
6. Implementare	37
6.1 Structura pachetelor	37

6.2	Layerul de prezentare	38
6.2.1	PatientController	38
6.2.2	DoctorController	39
6.2.3	AdminController	40
6.2.4	MessagingController	41
6.2.5	PaymentController și StripeController	41
6.2.6	ClinicApiController	42
6.3	Layerul de business	43
6.3.1	PatientService	43
6.3.2	DoctorService	44
6.3.3	AiService	45
6.3.4	StripeService	46
6.3.5	NotificationService	47
6.3.6	SmartQueueService	48
6.4	Layerul de acces la date	49
6.5	Securitatea aplicației	51
6.6	Notificările în timp real	53
6.7	Integrările externe	54
7.	Testare și instalare	56
7.1	Testarea automată	56
7.2	Testarea manuală	58
7.3	Instalarea locală	60
7.4	Instalarea cu Docker	61
7.5	Deploy în Google Cloud Run	62
8.	Concluzii și implementări ulterioare	64
9.	Bibliografie	67

Tabel de figuri

Figura 1. Schema generală a sistemului MediClinic	9
Figura 2. Arhitectura layered a aplicației Spring Boot	27
Figura 3. Diagrama use-case a sistemului	29
Figura 4. Diagrama de clase a entităților principale	31
Figura 5. Diagrama de secvență pentru fluxul de programare	33
Figura 6. Diagrama de stare pentru entitatea Appointment	34
Figura 7. Schema bazei de date (ERD)	36
Figura 8. Fragment de cod: configurarea Spring Security	51
Figura 9. Fragment de cod: rate limiter sliding-window	52
Figura 10. Fragment de cod: validarea webhook Stripe	54
Figura 11. Fragment de cod: analiza simptomelor cu fallback	55
Figura 12. Pagina de dashboard a administratorului	58
Figura 13. Pagina de programare a pacientului	59
Figura 14. Pagina de plată cu Stripe Checkout	59
Figura 15. Arhitectura de deploy în Google Cloud Run	62

1. Sinteza lucrării

Lucrarea de față descrie proiectul MediClinic, o aplicație web pentru managementul unei clinici medicale. Ideea a pornit de la observația că majoritatea clinicilor private din România folosesc încă instrumente fragmentate — Excel pentru programări, telefon pentru confirmări, hârtie pentru fișe medicale. Mi-am propus să construiesc o singură aplicație care să acopere tot fluxul: de la momentul în care pacientul își creează un cont, până la rețeta digitală pe care o primește după consultație.

Aplicația e construită în Java 21 cu Spring Boot 4, are MySQL ca bază de date și folosește Thymeleaf pentru paginile web. Pentru părțile mai speciale am integrat și câteva servicii externe: Stripe pentru plăți, DeepSeek (un model de limbaj similar cu ChatGPT) pentru analiza simptomelor descrise de pacient, Twilio pentru SMS și un server SMTP pentru emailuri. Toate aceste servicii sunt opționale — dacă nu sunt configurate, aplicația rulează în mod demonstrativ, util pentru testare.

Există trei tipuri de utilizatori. Pacientul își face cont, alege un doctor după specialitate, descrie simptomele și plătește taxa de rezervare. Doctorul vede agenda zilei, confirmă sau anulează programări, scrie fișe și rețete. Administratorul are un dashboard cu indicatori (câți pacienți, câte programări pe lună, venituri) și grafice generate cu Chart.js.

Securitatea a fost una dintre zonele unde am insistat cel mai mult. Parolele sunt hash-uite cu BCrypt, formularele POST au protecție CSRF, iar pagina de login e protejată împotriva atacurilor de tip brute-force printr-un rate limiter pe care l-am scris de la zero (sliding window, maxim 5 încercări pe minut per IP). Tot pentru securitate, webhook-urile Stripe sunt validate criptografic prin semnătură HMAC-SHA256.

Pentru a fi sigur că ce am scris funcționează, am acoperit codul cu 99 de teste automate — unele unitare cu Mockito, altele de integrare care simulează fluxul complet de booking. Tot ansamblul rulează în mai puțin de un minut și se execută automat la fiecare push pe GitHub prin GitHub Actions.

Aplicația poate fi pornită local cu Maven, în containere prin Docker Compose, sau direct în cloud pe Google Cloud Run. Pentru ultimul caz am pregătit și un ghid separat de deploy.

Pe parcurs am întâmpinat și am rezolvat câteva probleme interesante — de la `LazyInitializationException` la randarea template-urilor Thymeleaf, până la încercarea de a configura corect un Content Security Policy care să permită Stripe și să blocheze restul. Le discut în detaliu în capitolele care urmează

2. Introducere

2.1 Scopul

Obiectivul proiectului a fost realizarea unei aplicații web cu potențial de utilizare reală — adică să nu rămână doar un proiect academic, ci ceva care, cu mici ajustări, ar putea fi pus în producție într-o clinică reală. Domeniul medical a fost ales deoarece în România programările la doctor implică încă procese manuale: telefoane, foi de calcul, fișe pe hârtie.

Concret, aplicația trebuie să îndeplinească trei funcții principale. În primul rând, să permită pacientului să se programeze online, după specialitate sau direct la un doctor anume, plătind taxa de rezervare cu cardul. În al doilea rând, să ofere doctorului o agendă curată, cu fiecare pacient și simptomele sale, plus posibilitatea de a scrie fișe medicale și rețete care se generează ca PDF. Și în al treilea rând, să ofere administratorului un dashboard cu cifre — câți pacienți, câte programări, câte anulări, ce venituri — astfel încât deciziile să fie informate.

Pe lângă funcționalitățile de bază, s-au integrat câteva caracteristici moderne: analiza AI a simptomelor (pacientul scrie "mă doare în piept" și aplicația setează automat prioritatea pe CRITICAL și sugerează cardiologie), notificări în timp real prin WebSocket (când doctorul confirmă, pacientul vede instant un toast în browser) și procesare de plăți cu Stripe.

2.2 Motivație

Motivele pentru care a fost aleasă această temă sunt multiple.

În primul rând, obiectivul a fost aprofundarea cunoștințelor despre Spring Boot. La cursul de Tehnologii Web au fost parcurse noțiuni introductive, dar framework-ul este utilizat extensiv în industrie — la marile bănci, la companiile de telecom, la majoritatea startupurilor care realizează aplicații enterprise — iar un proiect de amploare s-a dovedit a fi cea mai bună metodă de înțelegere aprofundată.

Un alt motiv important l-a constituit aspectul de securitate. Proiectele mai mici de la școală nu implică date sensibile. Aici, însă, se lucrează cu date medicale și date de plată, deci implementarea corectă a fost esențială: hash-uire BCrypt pe parole, CSRF pe formulare, rate limiting pe login, validare criptografică pe webhook-urile Stripe, headere OWASP.

2.3 Importanța proiectului

Mediul medical din Romania e în plin proces de digitalizare, dar cu un decalaj clar față de Europa de Vest. Conform Eurostat, peste 60% din clinicile private din UE foloseau în 2023 măcar un sistem electronic de programări [27]. La noi, multe clinici private mici și mijlocii încă merg pe Excel și telefon. Asta înseamnă: programări dublate, pacienți care uită că au programare pentru că nu primesc reminder, timp pierdut cu apeluri telefonice repetate.

Pentru pacient, asta se traduce într-o experiență frustrantă: sună de mai multe ori să prindă un slot, nu știe sigur dacă programarea e confirmată, nu are acces ușor la istoricul lui medical. Pentru doctor, înseamnă timp pierdut cu administrație în loc de pacienți. Pentru admin, înseamnă rapoarte făcute manual la sfârșit de lună, cu greșeli inevitabile.

În plus, în ultimii doi ani modelele de limbaj de tip LLM (GPT, DeepSeek, Claude) au devenit suficient de bune încât pot fi folosite practic pentru triere medicală inițială. Nu vorbim de diagnostic — asta rămâne treaba doctorului — dar pot ajuta să direcționezi pacientul către specialistul potrivit. Un pacient care scrie “îmi e greu să respir și am dureri puternice în piept” ar trebui să ajungă la cardiolog cu prioritate, nu în coada generală.

MediClinic încearcă să rezolve aceste probleme într-o singură aplicație. Codul e suficient de modular încât o clinică reală ar putea, cu câteva zile de personalizare, să-l adopte. Iar pentru mine, ca student, e o platformă de pe care pot demonstra în practică o gamă largă de competențe — de la design de baze de date, până la deploy în cloud.

2.4 Structura proiectului

Sistemul MediClinic este organizat ca un monolit modular, urmând paradigma clasică de arhitectură stratificată (layered architecture). Aplicația este împărțită în patru layere logice principale, conform Figurii 1:

Layerul de prezentare primește request-urile HTTP de la browser, gestionează autentificarea și autorizarea, și pasează cererile mai departe către layerul de business.

Layerul de business logic conține regulile de aplicație și logica specifică domeniului medical. Aici se află serviciile dedicate fiecărei entități (PatientService, DoctorService, AppointmentService) și serviciile pentru integrările externe (StripeService, AiService, NotificationService).

Layerul de acces la date abstractizează interacțiunea cu baza de date relațională, folosind Spring Data JPA și Hibernate.

Layerul de persistență este reprezentat de baza de date MySQL 8, configurată cu charset utf8mb4 pentru suport complet pentru diacritice și caractere internaționale.

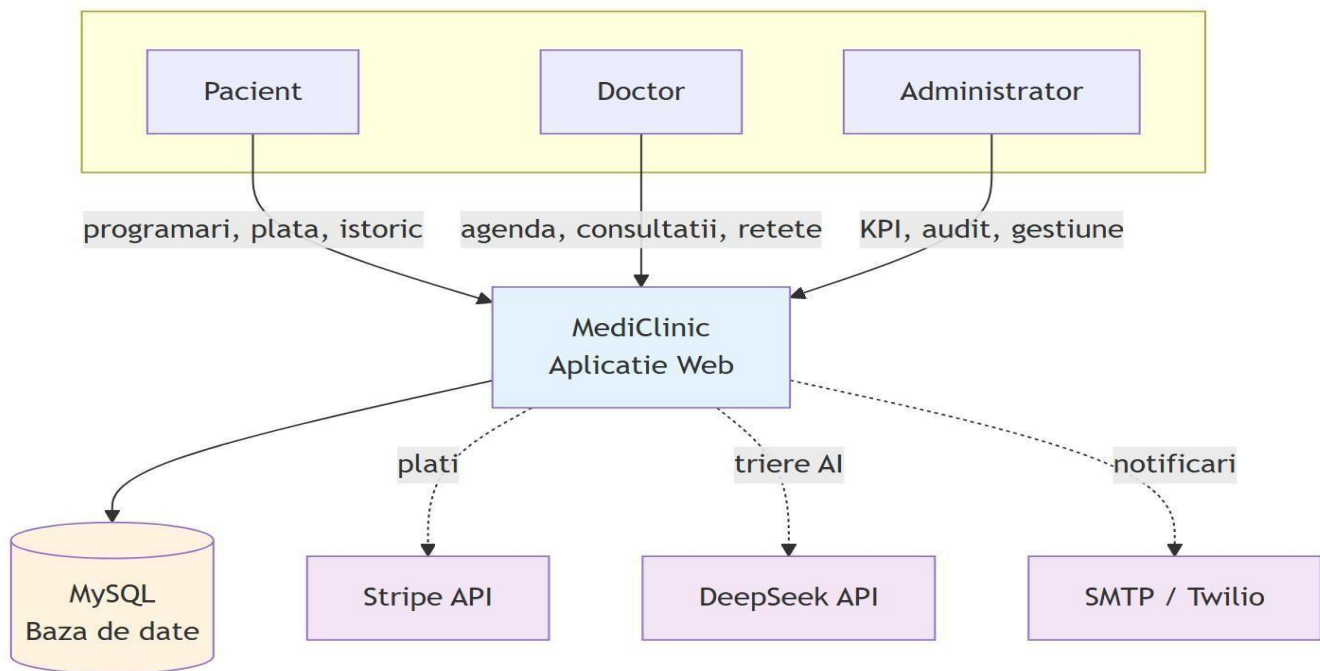


Figura 1. Schema generala a sistemului MediClinic

Astfel, lucrarea de față a fost împărțită în șapte capitole tehnice, urmărind structura recomandată pentru o lucrare de licență la Facultatea de Automatică și Calculatoare:

- **Capitolul 2** prezintă tehnologiile web și framework-urile folosite, justificând alegerile făcute pentru fiecare componentă a stack-ului tehnologic.
- **Capitolul 3** descrie cerințele funcționale și non-funcționale ale sistemului, împreună cu restricțiile tehnice impuse.
- **Capitolul 4** detaliază proiectarea aplicației prin diagrame UML (use-case, clase, secvență, stare) și prin schema relațională a bazei de date.
- **Capitolul 5** acoperă implementarea propriu-zisă, descriind clasele, serviciile și metodele esențiale din fiecare layer.
- **Capitolul 6** prezintă strategia de testare (automată și manuală) și pașii necesari pentru instalare și deploy.
- **Capitolul 7** concluzionează lucrarea, sintetizând rezultatele obținute și propunând direcții de dezvoltare viitoare.

3. Tehnologii web folosite

În acest capitol vor fi descrise tehnologiile, framework-urile și bibliotecile utilizate la dezvoltarea aplicației MediClinic. Pentru fiecare tehnologie se va prezenta scurt scopul ei, motivele alegerii și modul de integrare în arhitectura generală a sistemului.

3.1 Limbajul de programare Java

Java a fost ales ca limbaj principal datorită compatibilității cu Spring Boot și a maturității ecosistemului. La cursurile de POO și ISP s-a lucrat exclusiv în Java, ceea ce a permis concentrarea pe framework-uri și arhitectura aplicației.

Java este un limbaj orientat pe obiecte, dezvoltat inițial la Sun Microsystems în 1995 și menținut acum de Oracle. Ideea sa centrală — "write once, run anywhere" — vine din faptul că Java compilează la bytecode care rulează pe orice mașină cu JVM, indiferent de sistemul de operare. În practică, aceasta înseamnă că aplicația poate fi pornită pe Windows și mutată ca atare pe un server Linux în Google Cloud, fără modificări.

Câteva chestii noi din Java 21 pe care le-am folosit efectiv în proiect: - **Records** — pentru DTO-urile imutabile. În loc să scriu o clasă cu 5 getter-i, scriu un record într-o singură linie. - **Text blocks** — pentru prompt-urile multi-line pe care le trimit la DeepSeek. Mult mai citibil decât string-uri cu \n peste tot. - **Pattern matching** — în câteva locuri unde verific tipuri.

Pe **virtual threads** și **sealed classes** încă nu le-am folosit serios, dar sunt acolo dacă voi extinde aplicația mai târziu.

3.2 Framework-ul Spring Boot

Spring Boot e framework-ul peste care s-a construit toată aplicația. Pentru cine nu e familiarizat, Spring Boot e construit peste Spring Framework și aduce un concept simplu: în loc să configurezi tu manual sute de linii de XML sau Java pentru a porni o aplicație web (Tomcat, connection pool, security, JPA, etc.), Spring Boot face asta automat pe baza dependențelor pe care le declari în pom.xml. E ce se cheamă în jargon "convention over configuration".

Concret: am pus spring-boot-starter-data-jpa în pom.xml și automat am avut configurat HikariCP ca connection pool, EntityManager-ul Hibernate, transaction managerul. Nu am scris niciun fișier de config explicit pentru aceste lucruri. La fel cu spring-boot-starter-security — automat am avut un filter chain de bază, BCryptPasswordEncoder, formularul standard de login.

Am folosit **Spring Boot 4.0.5**, una dintre primele versiuni stabile din linia 4.x (lansată în 2025). Include Spring Framework 7 și Spring Security 7. Față de versiunile 3.x, principala diferență e că impune Java 17 minim și are câteva schimbări mici de API la Spring Security [9].

Lucrurile care mi-au plăcut cel mai mult la Spring Boot pe parcurs: - **Autoconfigurare** — mia economisit zile de muncă de configurare. - **Tomcat embedded** — nu trebuie instalat separat. Aplicația vine cu propriul server intern. - **Profiluri** — am application.properties pentru local,

application-cloud.properties pentru Google Cloud Run, applicationtest.properties pentru teste. Activez profilul corect cu o variabilă de mediu. - **Actuator** — endpoint-uri gata făcute pentru monitorizare. /actuator/health îmi spune dacă DBul răspunde, /actuator/metrics îmi arată câte programări s-au creat.

Sincer, după ce am încercat să fac de la zero o aplicație web în Java fără Spring (pentru un proiect anterior), apreciez cu adevărat cât de mult timp îmi economisește framework-ul ăsta.

3.3 Spring Security

Spring Security este modulul de securitate al ecosistemului Spring, oferind o soluție comprehensivă pentru autentificare, autorizare și protecție împotriva atacurilor web frecvente. Pentru acest proiect am folosit versiunea **7.0**, care vine implicit cu Spring Boot 4.0.5 [10].

Configurarea securității este centralizată în clasa SecurityConfig, care declară un bean de tip SecurityFilterChain. Aceasta definește:

- **Autentificarea prin form login** - utilizatorii își introduc email-ul și parola într-un formular HTML standard. La submit, request-ul este interceptat de Spring Security, parola este verificată împotriva hash-ului stocat în baza de date (folosind BCrypt), iar la succes se creează o sesiune HTTP.
- **Autorizare bazată pe roluri (RBAC)** - regulile sunt definite prin metoda authorizeHttpRequests. URL-urile care încep cu /admin/** necesită rolul ADMIN, cele cu /doctor/** necesită DOCTOR, iar cele cu /patient/** necesită PATIENT.
- **Protecție CSRF** - activată implicit pentru toate request-urile POST/PUT/DELETE, cu o singură excepție: webhook-ul Stripe, care este validat alternativ prin verificarea semnăturii criptografice.
- **Headere de securitate** - X-Frame-Options pentru protecția împotriva clickjackingului, X-Content-Type-Options pentru prevenirea MIME sniffing-ului, Referrer-Policy pentru limitarea informațiilor scurse prin Referer.

3.4 Spring Data JPA și Hibernate

Spring Data JPA este o componentă a ecosistemului Spring care abstractizează interacțiunea cu bazele de date relaționale, oferind o implementare elegantă a pattern-ului Repository. În spatele său, Spring Data JPA folosește Hibernate ca implementare a specificației Jakarta Persistence API (JPA) [11], [12].

În aplicația MediClinic, fiecare entitate JPA este o clasă Java anotată cu @Entity, cu câmpuri care reflectă coloanele tabelului din baza de date. Relațiile dintre entități sunt exprimate prin

anotările @OneToOne, @OneToMany, @ManyToOne și @ManyToMany, iar Hibernate generează automat join-urile SQL necesare.

Pentru fiecare entitate, am definit o interfață care extinde JpaRepository<Entitate, Long>. Spring Data implementează automat metodele standard (save, findById, findAll, delete), iar pentru query-uri custom am folosit anotarea @Query cu JPQL (Java Persistence Query Language).

O decizie importantă de design a fost dezactivarea modului “Open Session in View” prin setarea spring.jpa.open-in-view=false. Această alegere previne anti-pattern-ul în care sesiunea Hibernate rămâne deschisă pe toată durata request-ului, dar a impus folosirea explicită a JOIN FETCH în query-urile care încarcă entități legate prin relații lazy.

3.5 MySQL

MySQL este un sistem de gestiune a bazelor de date relaționale (RDBMS) cu sursă deschisă, dezvoltat inițial de MySQL AB și deținut acum de Oracle Corporation. Aplicația MediClinic folosește **MySQL versiunea 8.0** [14].

Alegerea MySQL în detrimentul altor RDBMS (PostgreSQL, MariaDB) s-a făcut pe baza câtorva considerente practice:

- **Răspândire largă** - MySQL este unul dintre cele mai folosite RDBMS în lume, cu suport excelent în Google Cloud Run (prin Cloud SQL), AWS (RDS) și alte platforme cloud.
- **Compatibilitate cu XAMPP** - pentru testarea locală, XAMPP oferă o instalare rapidă a MySQL împreună cu phpMyAdmin, util pentru explorarea datelor în timpul dezvoltării.
- **Charset utf8mb4** - configurat pentru suport complet la caracterele Unicode (inclusiv emoji și diacritice), esențial pentru o aplicație în limba română.

Schema bazei de date conține 15 tabele principale, cu integritate referențială asigurată prin chei străine. Indexurile sunt definite implicit pe coloanele de tip cheie primară și pe emailul utilizatorilor (unique).

3.6 Thymeleaf

Thymeleaf este un motor de șabloane Java modern, conceput special pentru integrarea cu Spring și pentru procesarea de fișiere HTML5 [15]. Spre deosebire de alte motoare de șabloane care folosesc sintaxă proprietară, Thymeleaf adoptă principiul “natural templates” - fișierele rămân HTML valid și pot fi vizualizate într-un browser chiar și fără server, ceea ce este util pentru designeri.

În aplicația MediClinic am folosit Thymeleaf 3 pentru a randa toate paginile dinamice ale interfeței web. Atributele Thymeleaf (th:text, th:if, th:each, th:href) permit injectarea de date din modelul Spring direct în HTML.

Exemplu de utilizare:

```
<table>
  <tr th:each="appt : ${appointments}">
    <td th:text="${appt.patient.fullName}">Nume pacient</td>
    <td th:text="${#temporals.format(appt.appointmentDate, 'dd MMM yyyy')}">< /td>
  </tr>
</table>
```

Integrarea Thymeleaf cu Spring Security este de asemenea facilitată, prin dialect-ul springsecurity care oferă atribute speciale pentru afișarea condiționată în funcție de rolul utilizatorului autentificat.

3.7 Bootstrap 5 și Chart.js

Pentru stilizarea componentelor UI și pentru obținerea unui design responsive (adaptabil la dimensiunile diferite de ecran) am folosit framework-ul **Bootstrap 5.3** [16]. Bootstrap oferă un sistem de grid bazat pe flexbox, componente pre-stilizate (carduri, modale, formulare, butoane) și utilitare CSS pentru spațiere, culori și tipografie.

Pentru reprezentarea datelor statistice pe dashboard-ul administratorului, am integrat biblioteca **Chart.js 4.4**, care oferă suport pentru grafice de tip linie, bară, plăcintă (doughnut) și radar. Graficele afișate în aplicație includ: programări pe lună (line chart), venituri pe lună (bar chart), ore aglomerate (line chart) și distribuția programărilor pe status (doughnut chart).

3.8 WebSocket și STOMP

Pentru notificările în timp real - de exemplu, anunțarea instantanee a pacientului când doctorul confirmă o programare - am folosit **WebSocket** cu protocolul **STOMP** (Simple Text Oriented Messaging Protocol) [25].

WebSocket este un protocol de comunicare full-duplex peste o singură conexiune TCP, standardizat prin RFC 6455. Spre deosebire de polling-ul HTTP clasic, WebSocket permite serverului să trimită mesaje către client fără ca acesta să facă cerere prealabilă, ceea ce reduce semnificativ latența pentru notificările push.

STOMP este un protocol de mesagerie text-based, simplu de implementat, care funcționează peste WebSocket. Spring Boot include un broker STOMP simplu (in-memory) prin starter-ul spring-boot-starter-websocket. Brokerul mai sofisticat (RabbitMQ, ActiveMQ) ar fi necesar doar pentru aplicații cu trafic foarte mare; pentru o clinică tipică, brokerul in-memory este suficient.

În aplicație, fiecare utilizator autentificat se abonează automat la canalul /user/queue/notifications la deschiderea unei pagini. Când un eveniment relevant se întâmplă (programare nouă, confirmare, anulare), serverul publică un mesaj pe acest canal și browserul utilizatorului afișează o notificare instant.

3.9 Stripe API

Stripe este unul dintre cele mai răspândite procesatoare de plăți online la nivel global, oferind un API REST modern și SDK-uri oficiale pentru toate limbajele populare. Aplicația MediClinic folosește SDK-ul oficial **stripe-java versiunea 26.3.0** [17].

Modul de operare este următorul: când pacientul dorește să confirme o programare, aplicația apelează Stripe pentru a crea o “Checkout Session” - un obiect care reprezintă o intenție de plată. Stripe returnează un URL temporar (valabil ~24 ore) către care pacientul este redirectat. Acolo, pacientul introduce datele cardului direct pe domeniul Stripe (deci datele de card NU trec niciodată prin aplicația MediClinic, ceea ce reduce drastic complianța PCIDSS).

După finalizarea plății, Stripe trimite un webhook către endpoint-ul /stripe/webhook al aplicației, cu detaliile evenimentului. Webhook-ul este securizat prin verificarea semnăturii criptografice (header Stripe-Signature), asigurându-se astfel că request-ul provine cu adevărat de la Stripe.

Pentru a permite rularea aplicației în mod demonstrativ fără cont Stripe real, am implementat un fallback prin proprietatea payment.demo-fallback-enabled=true. În acest mod, plata este simulată local și statusul este marcat ca succes fără apel HTTP către Stripe.

3.10 DeepSeek AI

DeepSeek este o platformă chineză de modele de limbaj (Large Language Models) care oferă un API compatibil cu standardul OpenAI. Modelul deepseek-chat este folosit în aplicație pentru două scopuri principale:

- **Recomandarea specialității medicale** pe baza simptomelor descrise în limbaj natural de pacient.
- **Analiza urgenței simptomelor** și clasificarea lor în patru niveluri (CRITICAL, HIGH, MEDIUM, LOW).

Apelul către API-ul DeepSeek se face prin clasa HttpClient standard din Java 11+, fără a depinde de un SDK extern. Request-ul este un POST către <https://api.deepseek.com/chat/completions>, cu un body JSON ce conține mesajele (system prompt + user prompt) și parametrii modelului.

Pentru a obține răspunsuri consistente, am construit un system prompt scurt și restrictiv:

You are a medical triage assistant. Return exactly one word from this list: cardiology, dermatology, neurology, general, pediatrics, ... No explanation. No punctuation. Lowercase only.

Astfel, modelul este constrâns să returneze doar una din specialitățile disponibile, evitând răspunsuri free-form care ar fi greu de parsat programatic.

În cazul în care API-ul nu este configurat (cheie API lipsă) sau nu răspunde în timp util (timeout 10s), aplicația revine la o euristică bazată pe cuvinte cheie, definită explicit în codul AiServiceImpl. Această euristică acoperă cazurile critice (durere în piept → CRITICAL → cardiologie, paralizie → CRITICAL → neurologie, etc.).

3.11 OpenPDF

OpenPDF este un fork open-source al bibliotecii iText, distribuit sub licență LGPL/MPL. Spre deosebire de versiunile mai noi ale iText (5.5+) care necesită licență comercială pentru utilizare în software proprietar, OpenPDF rămâne complet gratuit. În aplicație folosim **versiunea 1.3.43**.

OpenPDF este utilizat pentru generarea programatică a două tipuri de documente PDF:

- **Facturi** - emise automat la finalizarea unei plăți. Conțin antetul MediClinic, datele pacientului, detaliile programării și suma plătită.
- **Rețete medicale** - generate de doctor după consultație. Conțin numele pacientului, diagnoză, medicamentele prescrise și instrucțiunile de administrare.

PDF-urile sunt construite prin instanțierea unui obiect Document, adăugarea de elemente

(Paragraph, Table, Chunk) și apoi serializate într-un ByteArrayOutputStream care este returnat ca răspuns HTTP cu content-type application/pdf.

3.12 Twilio SMS API

Pentru notificările prin SMS am integrat platforma **Twilio**, lider mondial în servicii de comunicații programabile. Twilio oferă un SDK oficial pentru Java (versiunea 10.6.3 în acest proiect) care abstractizează apelurile REST către API-ul lor.

Trimiterea unui SMS se reduce la trei linii de cod:

```
Twilio.init(accountSid, authToken);  
Message.creator(new PhoneNumber("+40721234567"), new PhoneNumber(twilioFromPhone), "Mesaj").create();
```

Numerele de telefon sunt convertite automat din formatul românesc (07xx...) în formatul internațional E.164 (+407xx...) prin metoda privată toE164() din NotificationServiceImpl. Această conversie este necesară deoarece Twilio acceptă doar numere în format internațional.

Costurile pentru SMS variază în funcție de țara destinatarului (aproximativ 0.04 USD per SMS pentru România), ceea ce face viabilă utilizarea în producție doar pentru notificările esențiale (programare confirmată, reminder cu 24h înainte).

3.13 Docker și Docker Compose

Docker este o platformă de containerizare care permite împachetarea aplicațiilor cu toate dependențele lor într-un format standardizat, executabil pe orice mașină care are Docker instalat. Aplicația MediClinic include un Dockerfile cu un build multi-stage:

- **Stage-ul de build** folosește imaginea eclipse-temurin:21-jdk-alpine, copiază sursele și execută mvn package.
- **Stage-ul de runtime** folosește imaginea mai compactă eclipse-temurin:21-jre-alpine și execută JAR-ul rezultat.

Multi-stage build-ul rezultă într-o imagine finală de aproximativ 200 MB, semnificativ mai mică decât dacă am fi inclus JDK-ul complet (~400 MB).

Pentru orchestrarea aplicației împreună cu serviciile dependente (MySQL, phpMyAdmin), folosim **Docker Compose**. Fișierul docker-compose.yml definește trei servicii: aplicația Spring Boot, baza de date MySQL 8 și un container phpMyAdmin pentru administrare. Toate parolele sunt externalizate într-un fișier .env (gitignored).

3.14 JUnit 5 și Mockito

Testarea automată este realizată cu **JUnit Jupiter (JUnit 5)**, framework-ul standard pentru testarea unitară în Java modern. JUnit 5 oferă anotări declarative (@Test, @BeforeEach, @DisplayName), suport pentru teste parametrizate și extensii care permit integrarea cu alte biblioteci.

Pentru mock-uirea dependențelor în testele unitare folosim **Mockito**, biblioteca dominantă pentru această sarcină în ecosistemul Java. Anotările @Mock și @InjectMocks, împreună cu extensia MockitoExtension, simplifică drastic scrierea testelor.

Pentru aserțiuni mai expresive folosim biblioteca **AssertJ**, care oferă un API fluent (`assertThat(result).isEqualTo(expected)`) și mesaje de eroare detaliate.

Suita de testare include 99 de teste, distribuite astfel: - 24 teste pentru `AiServiceImpl` - 14 teste pentru `PatientServiceImpl` - 12 teste pentru `DoctorServiceImpl` - 11 teste pentru `AppointmentRepository` (cu H2 in-memory) - 17 teste pentru securitate (RBAC, CSRF, redirecturi) - 10 teste pentru integrare (`BookingFlowIntegrationTest`) - 6 teste pentru schedulerul de cleanup - 4 teste pentru scheduler-ul de reminder - 1 test de smoke pentru pornirea contextului Spring

3.15 Maven

Apache Maven este unealta de build și management al dependențelor folosită în acest proiect. Versiunea wrapped (mvnw) este inclusă în repository, permițând rularea fără a fi nevoie de instalare globală.

Fișierul pom.xml declară: - Părinte spring-boot-starter-parent versiunea 4.0.5 - Versiunea Java 21 - Lista de dependențe (Spring Boot starters, Stripe, Twilio, OpenPDF, Plugin-uri (maven-compiler-plugin, spring-boot-maven-plugin)

Comenzile uzuale folosite în dezvoltare sunt mvnw spring-boot:run (rulează aplicația), mvnw test (rulează testele) și mvnw package -DskipTests (construiește JAR-ul executabil).

3.16 SpringDoc OpenAPI

Pentru generarea automată a documentației REST API folosim **SpringDoc OpenAPI versiunea 2.8.8**, o bibliotecă care scanează adnotările `@RestController` și `@Operation` din cod și produce o specificație OpenAPI 3.0 conformă.

Documentația interactivă este disponibilă la URL-ul `/swagger-ui/index.html`. Aici, dezvoltatorii și testerii pot vedea toate endpoint-urile, parametrii lor, corpul de request/response și pot rula apeluri de test direct din browser.

Configurarea suplimentară a documentației (titlu, descriere, versiune, informații de contact) se face în clasa `OpenApiConfig`, prin definirea unui bean `OpenAPI`.

4. Cerințele și specificațiile proiectului

4.1 Cerințe funcționale

Cerințele funcționale descriu CE trebuie să facă aplicația - ce operațiuni concrete sunt expuse utilizatorilor finali. Pentru claritate, le-am împărțit pe cele trei roluri suportate de sistem.

4.1.1 Cerințe pentru rolul de pacient

- **CF-01:** Pacientul se poate înregistra în sistem furnizând: prenume, nume, email, parolă și telefon (opțional).
- **CF-02:** Pacientul se poate autentifica folosind email-ul și parola.
- **CF-03:** În cazul uitării parolei, pacientul poate solicita un link de resetare trimis prin email.
- **CF-04:** Pacientul poate vizualiza lista doctorilor, cu posibilitatea de filtrare după specialitate.
- **CF-05:** Pacientul poate consulta profilul detaliat al unui doctor (specialitate, rating mediu, comentarii primite).
- **CF-06:** Pacientul poate crea o programare nouă, alegând doctorul, data și ora din sloturile disponibile.
- **CF-07:** La crearea programării, sistemul analizează automat simptomele descrise (prin DeepSeek AI sau fallback pe euristică) și stabilește prioritatea (LOW/MEDIUM/HIGH/CRITICAL).
- **CF-08:** Pacientul plătește taxa de rezervare prin Stripe Checkout (sau prin fallback demonstrativ).
- **CF-09:** Pacientul poate vizualiza istoricul propriu: programări trecute, fișe medicale, rețete și facturi.
- **CF-10:** Pacientul poate descărca facturile în format PDF.
- **CF-11:** Pacientul primește notificări instant prin WebSocket când doctorul confirmă, anulează sau finalizează programarea.
- **CF-12:** Pacientul poate evalua doctorul cu o notă de la 1 la 5 stele și un comentariu opțional, după ce consultația a fost finalizată.
- **CF-13:** Pacientul poate trimite și primi mesaje de la doctor prin sistemul de mesagerie integrat.

4.1.2 Cerințe pentru rolul de doctor

- **CF-14:** Doctorul vede agenda zilei, cu programările confirmate și cele în așteptare.
- **CF-15:** Doctorul poate confirma o programare în starea PENDING (după ce taxa de rezervare a fost plătită).
- **CF-16:** Doctorul poate anula o programare. Dacă taxa a fost plătită, sistemul declanșează automat o rambursare prin Stripe

- **CF-17:** Doctorul poate marca o programare confirmată finalizată, salvând notele consultației.
- **CF-18:** Doctorul vede o “coadă inteligentă” cu pacienții programați azi, ordonați după scor de prioritate.
- **CF-19:** Doctorul poate adăuga o fișă medicală (simptome, diagnostic, tratament, observații) pentru fiecare consultație.
- **CF-20:** Doctorul poate scrie o rețetă cu medicamente și instrucțiuni de administrare, descărcabilă în PDF.
- **CF-21:** Doctorul vizualizează statistici personale: număr programări pe lună, rating mediu, listă rating-uri primite.
- **CF-22:** Doctorul își poate edita profilul (specialitate, telefon, bio).

4.1.3 Cerințe pentru rolul de administrator

- **CF-23:** Administratorul are acces la un dashboard cu indicatori KPI: total pacienți, total doctori, total programări, venituri încasate, facturi neachitate.
- **CF-24:** Administratorul vede grafice statistice: distribuția programărilor pe status, evoluția lunară a programărilor, evoluția lunară a veniturilor, orele cele mai aglomerate, top 5 doctori după număr de consultații.
- **CF-25:** Administratorul listează și gestionează toți utilizatorii din sistem.
- **CF-26:** Administratorul vede toate programările cu filtrare după status și căutare după pacient/doctor.
- **CF-27:** Administratorul accesează audit log-ul complet al acțiunilor importante din sistem.
- **CF-28:** Administratorul poate exporta lista programărilor în format CSV pentru rapoarte externe.

4.2 Cerințe non-funcționale

Cerințele non-funcționale descriu CALITĂȚI ale sistemului, dincolo de funcționalitățile concrete. Acestea includ securitatea, performanța, disponibilitatea și mentenabilitatea.

4.2.1 Cerințe de securitate

- **CNF-01:** Parolele utilizatorilor sunt stocate hash-uite cu algoritmul BCrypt (factor cost 10). Algoritmul BCrypt este recomandat pentru parole datorită caracteristicii sale de “slow hashing”, care îngreunează atacurile prin forță brută [24].
- **CNF-02:** Toate formularele care modifică starea (POST/PUT/DELETE) sunt protejate împotriva atacurilor CSRF prin token-uri unice generate per sesiune.
- **CNF-03:** Endpoint-ul /login este protejat împotriva atacurilor brute-force prin rate limiter sliding-window (maxim 5 încercări per IP per minut).
- **CNF-04:** Răspunsurile HTTP includ headere de securitate: X-

Frame-Options (anticlickjacking), X-Content-Type-Options (anti-MIME-sniffing), Referrer-Policy (limitare scurgere informații).

CNF-05: Webhook-ul Stripe validează criptografic semnătura request-ului folosind header-ul Stripe-Signature și webhook secret-ul configurat.

- **CNF-06:** Aplicația folosește RBAC cu trei roluri (PATIENT, DOCTOR, ADMIN). Tentativele de acces la resurse neautorizate sunt blocate cu HTTP 403 [27].
- **CNF-07:** Sesiunea HTTP expiră după 30 minute de inactivitate, eliminând riscul lăsării sesiunilor deschise pe calculatoare partajate.
- **CNF-08:** Cookie-ul de sesiune este marcat HttpOnly și SameSite=Lax, prevenind accesul din JavaScript și atacurile CSRF cross-site.

4.2.2 Cerințe de performanță

- **CNF-09:** Timpul de răspuns la 95% din request-urile non-cache este sub 500 milisecunde, pe un volum de date corespunzător unei clinici tipice (~10.000 programări, ~1.000 pacienți).
- **CNF-10:** Aplicația poate servi simultan minim 100 utilizatori concurenți pe o instanță cu 512 MB RAM și 1 vCPU.
- **CNF-11:** Notificările prin email și SMS sunt trimise asincron (cu @Async), pentru a nu bloca request-ul HTTP al utilizatorului.

4.2.3 Cerințe de disponibilitate

- **CNF-12:** Aplicația expune endpoint-ul /actuator/health care raportează starea bazei de date, a serviciilor externe și a disk space-ului. Acest endpoint este folosit de probele Kubernetes/Cloud Run pentru a verifica dacă instanța este sănătoasă.
- **CNF-13:** Notificările prin email și SMS sunt opționale. Lipsa configurării SMTP sau Twilio nu împiedică funcționarea aplicației.
- **CNF-14:** Integrarea cu DeepSeek AI are fallback automat pe euristică bazată pe cuvinte cheie, asigurând că nicio programare nu rămâne fără analiză de prioritate.

4.2.4 Cerințe de mentenabilitate

- **CNF-15:** Codul este organizat în layere clare (Controller, Service, Repository, Domain), facilitând înțelegerea și extensia ulterioară.
- **CNF-16:** Aplicația include o suită de teste automate de 99 de teste, care rulează în mai puțin de 60 secunde și acoperă toate fluxurile critice.
- **CNF-17:** Configurarea sensibilă (parole DB, chei API) este externalizată în variabile de mediu, urmând principiul 12-factor app [4].
- **CNF-18:** Documentația REST API este generată automat prin SpringDoc OpenAPI și accesibilă la /swagger-ui/index.html.

4.3 Restricții tehnice

Pentru rularea aplicației sunt necesare următoarele:

- **Java Runtime:** JDK 21 sau mai recent.
 - **Bază de date:** MySQL 8.0 sau MariaDB 10.5+ (compatibil prin driverul JDBC).
- Memorie RAM:** minim 512 MB disponibili pentru JVM.
- **Spațiu disk:** ~200 MB pentru JAR-ul aplicației + dependențe, plus spațiul ocupat de baza de date.
 - **Browser:** suport pentru JavaScript ES2015 și WebSocket (toate browserele moderne din ultimii 5 ani).
 - **Conexiune internet:** necesară pentru funcționalitățile de plată (Stripe), notificări (Twilio, SMTP) și analiză AI (DeepSeek). Aplicația rulează însă și fără internet, în mod demonstrativ.

5. Proiectarea

5.1 Schema bloc a aplicației

Aplicația MediClinic este proiectată ca un monolit modular, urmând arhitectura clasică stratificată (layered architecture). Această abordare separă responsabilitățile în patru layer distincte, fiecare comunicând doar cu layerul direct vecin, conform Figurii 2.

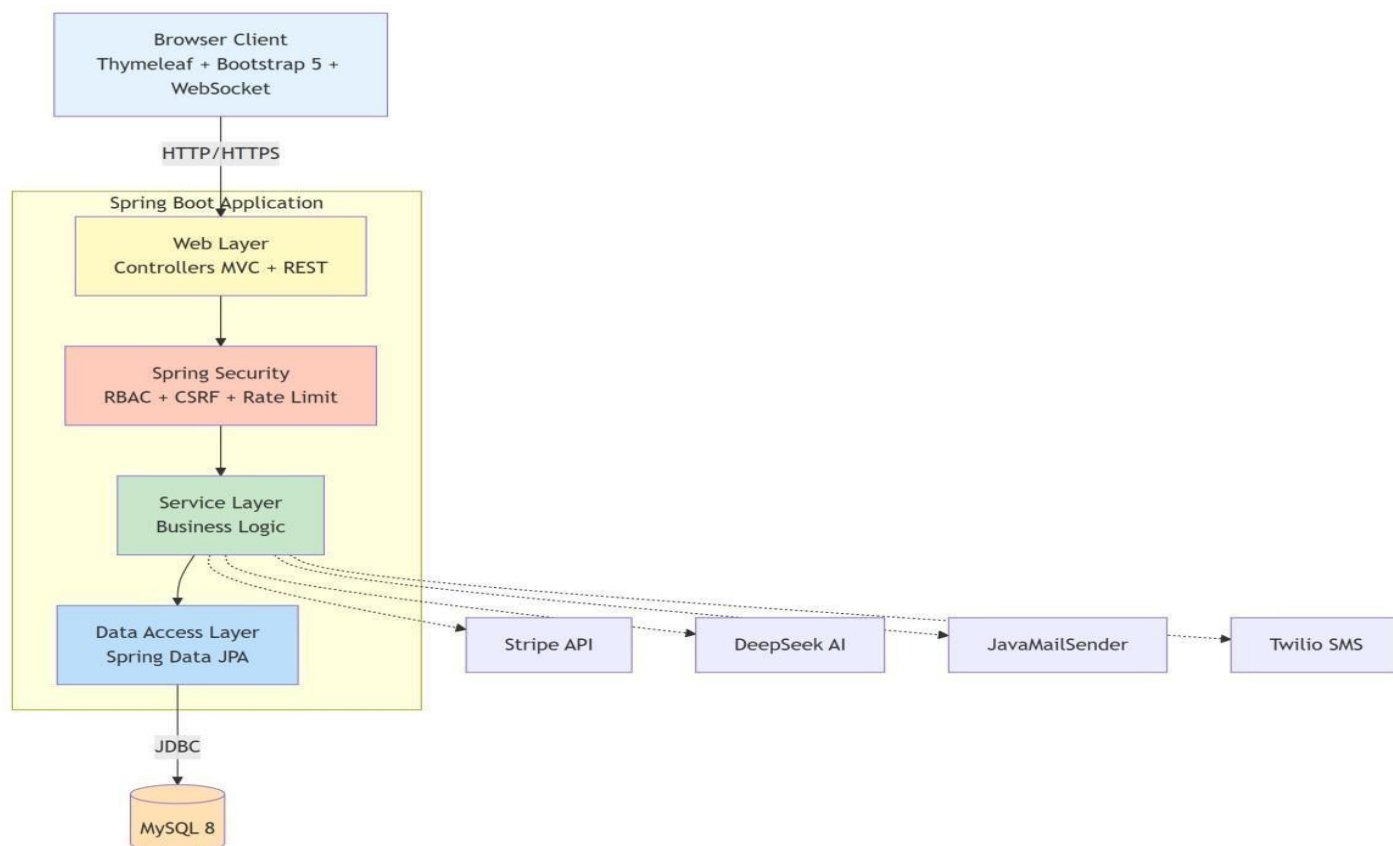


Figura 2. Arhitectura layered a aplicației Spring Boot

Layerul de prezentare (Web Layer) primește request-urile HTTP de la browser și le rutează către controller-ele corespunzătoare. În acest layer se află toate clasele anotate cu `@Controller` (pentru pagini Thymeleaf) și `@RestController` (pentru endpoint-uri JSON). Layerul tratează și parsarea parametrilor de query, validarea formurilor și serializarea răspunsurilor.

Layerul de securitate (Security Layer) este implementat prin Spring Security și interceptează toate request-urile înainte ca ele să ajungă la controller-e. Aici se verifică dacă utilizatorul este autentificat, dacă are rolul necesar pentru resursa cerută, dacă token-ul CSRF este valid (pentru POST-uri) și dacă a depășit limitele de rate limiting. Tot aici se adaugă headerele HTTP de securitate la răspuns.

Layerul de business logic (Service Layer) conține regulile aplicației. Serviciile sunt anotate cu `@Service` și injectate prin constructor în controller-e. Aici se află logica de programare a unei consultații (validare slot, analiză AI, salvare), de procesare a unei plăți Stripe, de generare a unui PDF de factură, etc. Tot aici se fac apelurile către serviciile externe (Stripe, DeepSeek, Twilio).

Layerul de acces la date (Data Access Layer) este implementat prin Spring Data JPA. Repository-urile sunt interfețe care extind `JpaRepository<Entitate, Long>` și sunt implementate automat de Spring la runtime. Pentru query-uri complexe folosim adnotarea `@Query` cu JPQL.

Comunicarea cu baza de date MySQL se face prin driver-ul oficial mysql-connectorj, folosind conexiuni pooled gestionate de HikariCP (configurat automat de Spring Boot).

5.2 Diagrama use-case

Diagrama use-case (Figura 3) prezintă funcționalitățile sistemului din perspectiva celor trei actori principali: Pacientul, Doctorul și Administratorul.

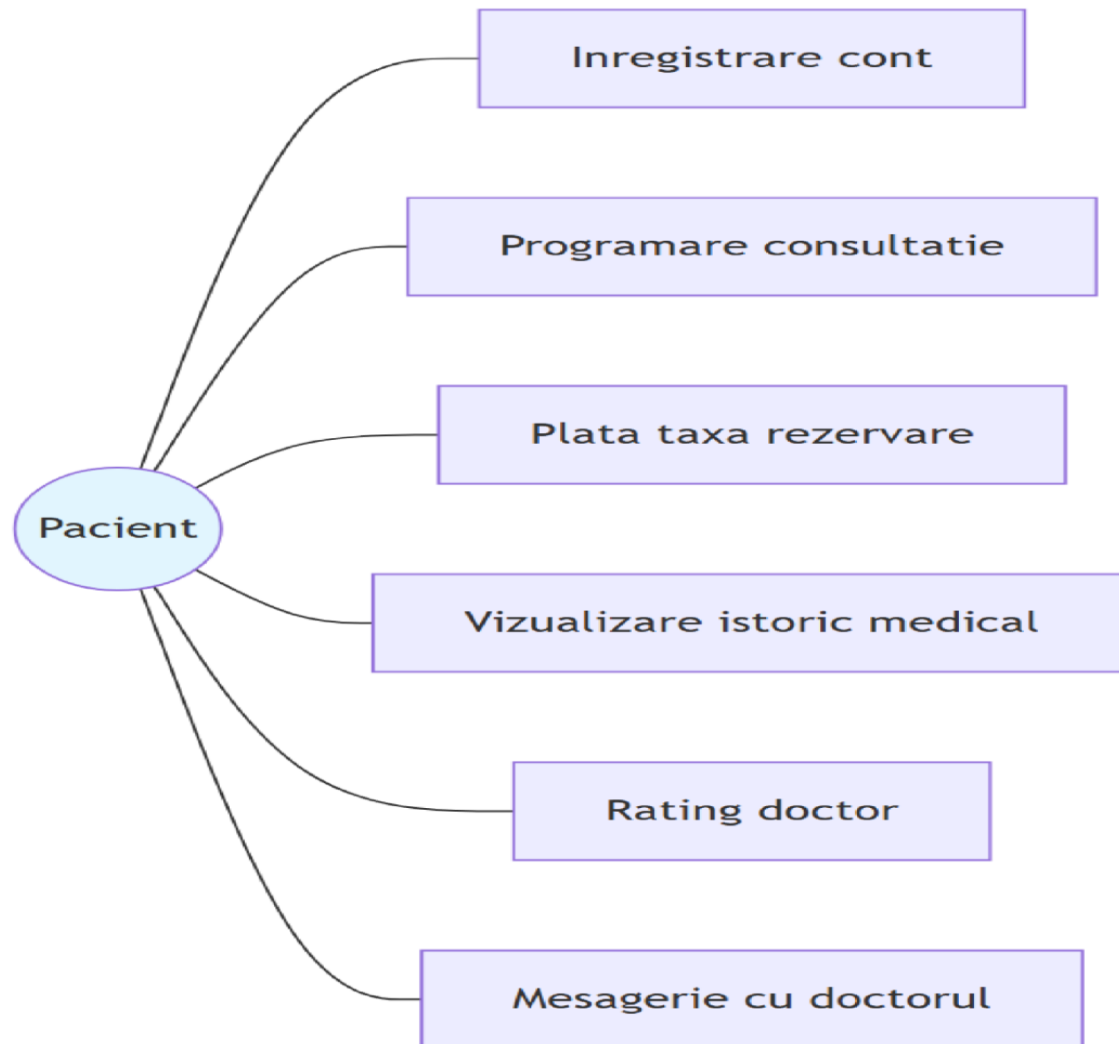


Figura 3. Diagrama use-case a sistemului

Diagrama evidențiază separarea clară a funcționalităților per rol, ceea ce se traduce direct în configurarea Spring Security: URL-urile `/patient/**` sunt accesibile doar pacienților, `/doctor/**` doar doctorilor, iar `/admin/**` doar administratorilor.

Există însă și funcționalități împărtășite implicit între roluri, cum ar fi mesageria internă (atât pacientul cât și doctorul pot trimite mesaje) sau modificarea profilului personal. Aceste funcționalități sunt expuse prin endpoint-uri separate pentru fiecare rol, păstrând separarea strictă a permisiunilor.

5.3 Diagrama de clase

Modelul de date al aplicației este reprezentat în Figura 4, sub forma unei diagrame UML de clase. În centrul modelului se află entitatea `User`, care reprezintă orice utilizator autentificat în sistem (indiferent de rol).

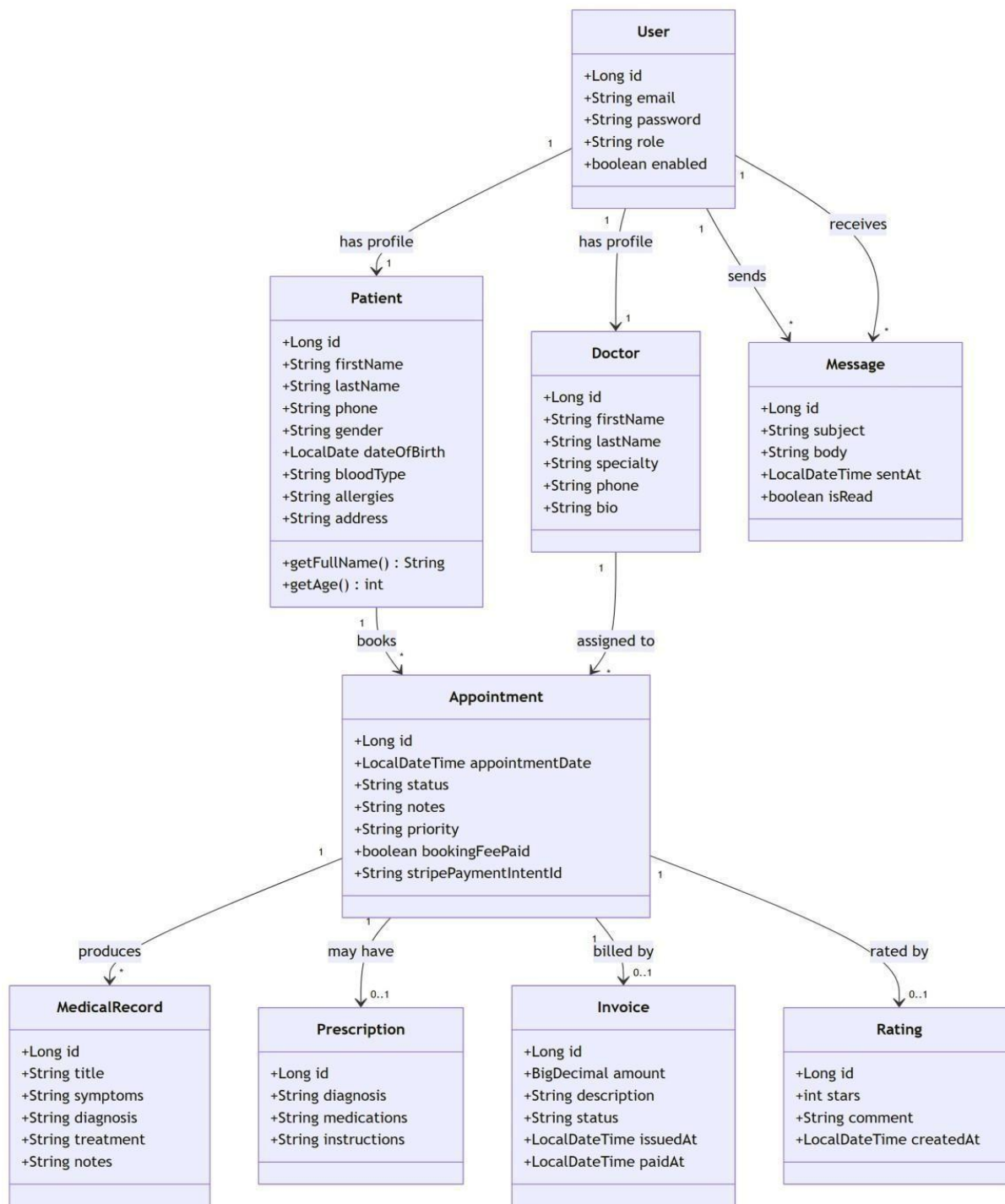


Figura 4. Diagrama de clase a entitatilor

principale Relațiile principale dintre entități sunt:

- User are o relație 1:1 opțională cu Patient (dacă rolul este PATIENT) sau cu Doctor (dacă rolul este DOCTOR). Administratorii nu au profil separat.
- Patient și Doctor au fiecare o relație 1:N cu Appointment (un pacient/doctor poate avea multe programări).

- Appointment are relații 1:N cu MedicalRecord (o programare poate produce mai multe fișe medicale, deși în practică e una) și 1:1 cu Invoice (factura asociată) și Rating (evaluarea de la pacient).
- Prescription se leagă atât de Patient, cât și de Doctor și de Appointment.
- Message are două relații cu User (sender și receiver) și o relație self-referential cu el însuși (pentru thread-uri de răspunsuri).

Toate entitățile au câmpul id de tip Long (BIGINT în MySQL), generat automat de baza de date prin AUTO_INCREMENT. Câmpurile temporale (createdAt, sentAt, paidAt) sunt de tip LocalDateTime din pachetul java.time (Java 8+).

5.4 Diagrama de secvență pentru fluxul de programare

Fluxul de programare a unei consultații este cel mai complex flux funcțional al aplicației, implicând opt actori: pacientul, browser-ul, controller-ul Spring, serviciul AI, baza de date, Stripe, webhook-ul, și serviciul de notificări (Figura 5).

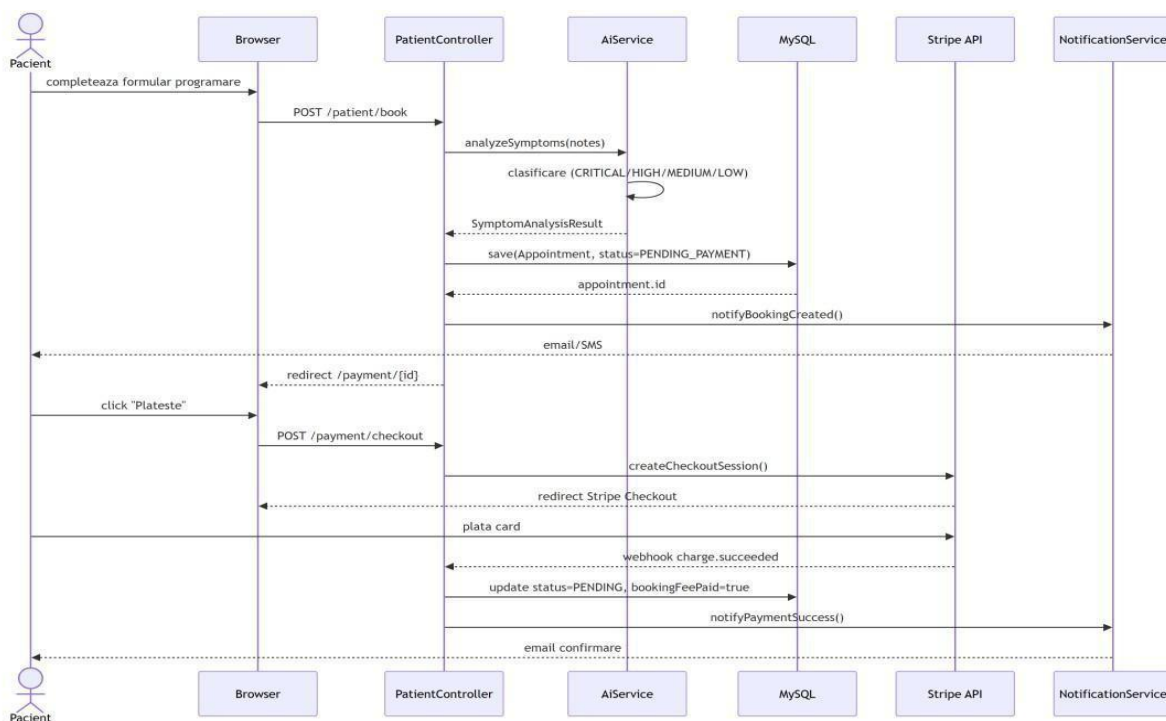


Figura 5. Diagrama de secvență pentru fluxul de programare

Pașii fluxului sunt:

1. **Pacientul completează formularul** de programare cu doctorul, data/ora și descrierea simptomelor.
2. **Browser-ul trimite POST** către /patient/book cu datele formularului.
3. **PatientController apelează AIService** pentru analiza simptomelor.
4. **AIService face request HTTP** către DeepSeek API (sau fallback pe euristică dacă API-ul nu e configurat).

5. **AIService** returnează un **SymptomAnalysisResult** cu specialitatea, prioritatea și motivul.
6. **PatientService** salvează **Appointment-ul** în starea **PENDING_PAYMENT**.
7. **NotificationService** trimite **email/SMS** către pacient pentru confirmare.
8. **Aplicația** redirectează **pacientul** către pagina de plată.
9. **Pacientul** plătește prin **Stripe Checkout** (sau prin fallback demonstrativ).
10. **Stripe** trimite **webhook** către `/stripe/webhook` cu evenimentul `checkout.session.completed`.
11. **StripeController** validează **semnătura** și actualizează statusul programării la **PENDING**.
12. **NotificationService** trimite **email de confirmare** pentru plata reușită.

Acest flux durează în practică între 30 secunde și 2 minute, în funcție de cât de rapid completează pacientul datele de card.

5.5 Diagrama de stare pentru Appointment

Entitatea Appointment parcurge o serie de stări bine definite pe parcursul vieții sale, conform Figurii 6.

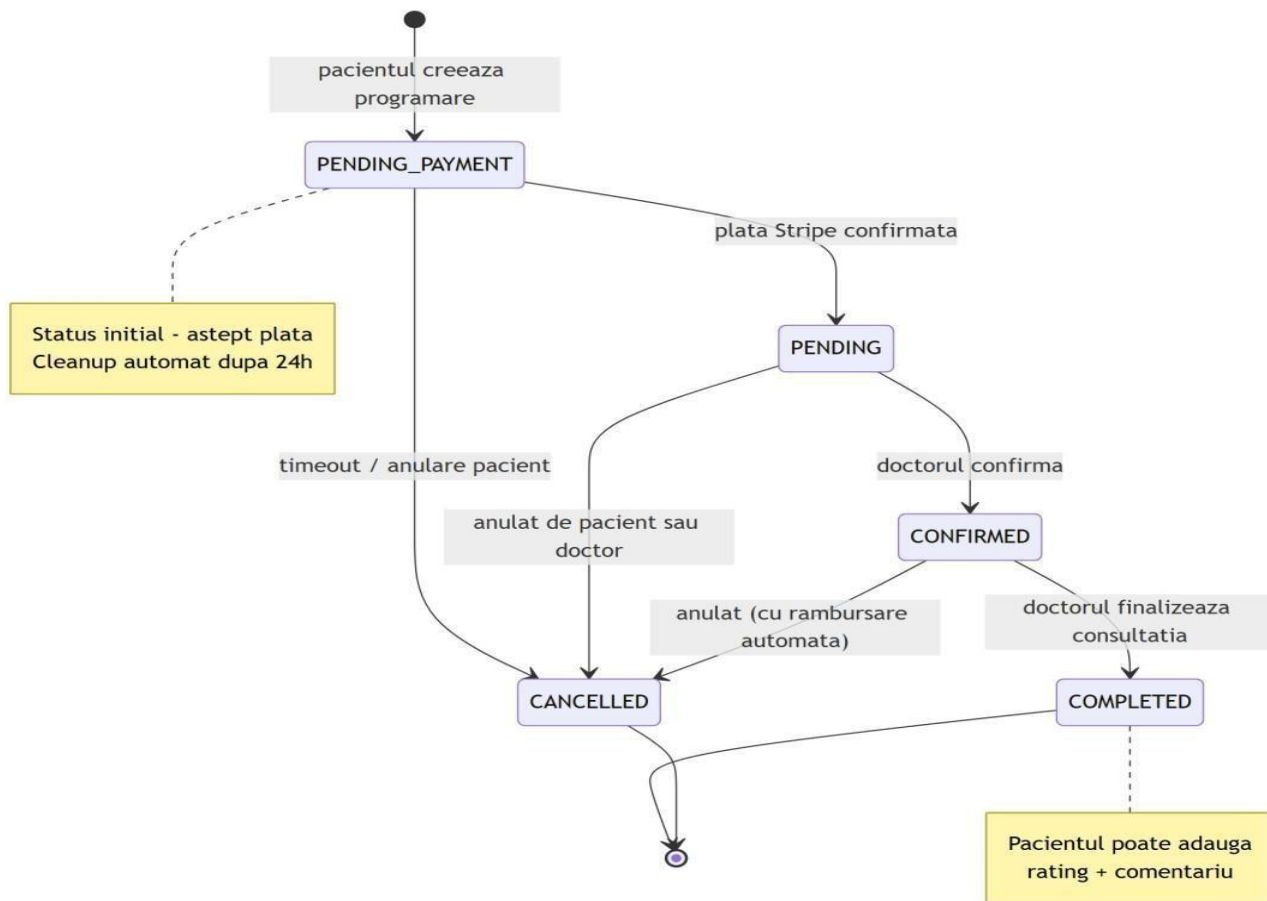


Figura 6. Diagrama de stare pentru entitatea

Appointment Stările sunt:

- **PENDING_PAYMENT** - starea inițială, imediat după crearea programării de pacient. Așteaptă plata.
- **PENDING** - taxa de rezervare a fost plătită. Așteaptă confirmarea din partea doctorului.
- **CONFIRMED** - doctorul a confirmat programarea. Pacientul este așteptat la consultație.
- **COMPLETED** - consultația a avut loc, doctorul a finalizat programarea și a salvat notele.
- **CANCELLED** - programarea a fost anulată (de pacient, de doctor sau automat după timeout).

Tranzițiile între stări sunt validate strict la nivelul service-ului. De exemplu, metoda `DoctorService.confirmAppointment()` aruncă o excepție dacă programarea nu este în starea **PENDING**. Această validare previne situații anormale precum confirmarea unei programări deja finalizate.

Un scheduler dedicat (`AppointmentCleanupTask`) rulează periodic și anulează automat programările care rămân în starea **PENDING_PAYMENT** mai mult de 24 ore. Această măsură previne acumularea de programări “moarte” care rezervă sloturi fără a fi plătite.

5.6 Schema bazei de date

Schema bazei de date (Figura 7) cuprinde 15 tabele principale, organizate pe trei niveluri logice: tabele de utilizatori (app_user, patient, doctor), tabele tranzacționale (appointment, invoice, prescription, medical_record, rating) și tabele auxiliare (audit_log, activity_log, message, app_notification, password_reset_token, medical_report, doctor_schedule).

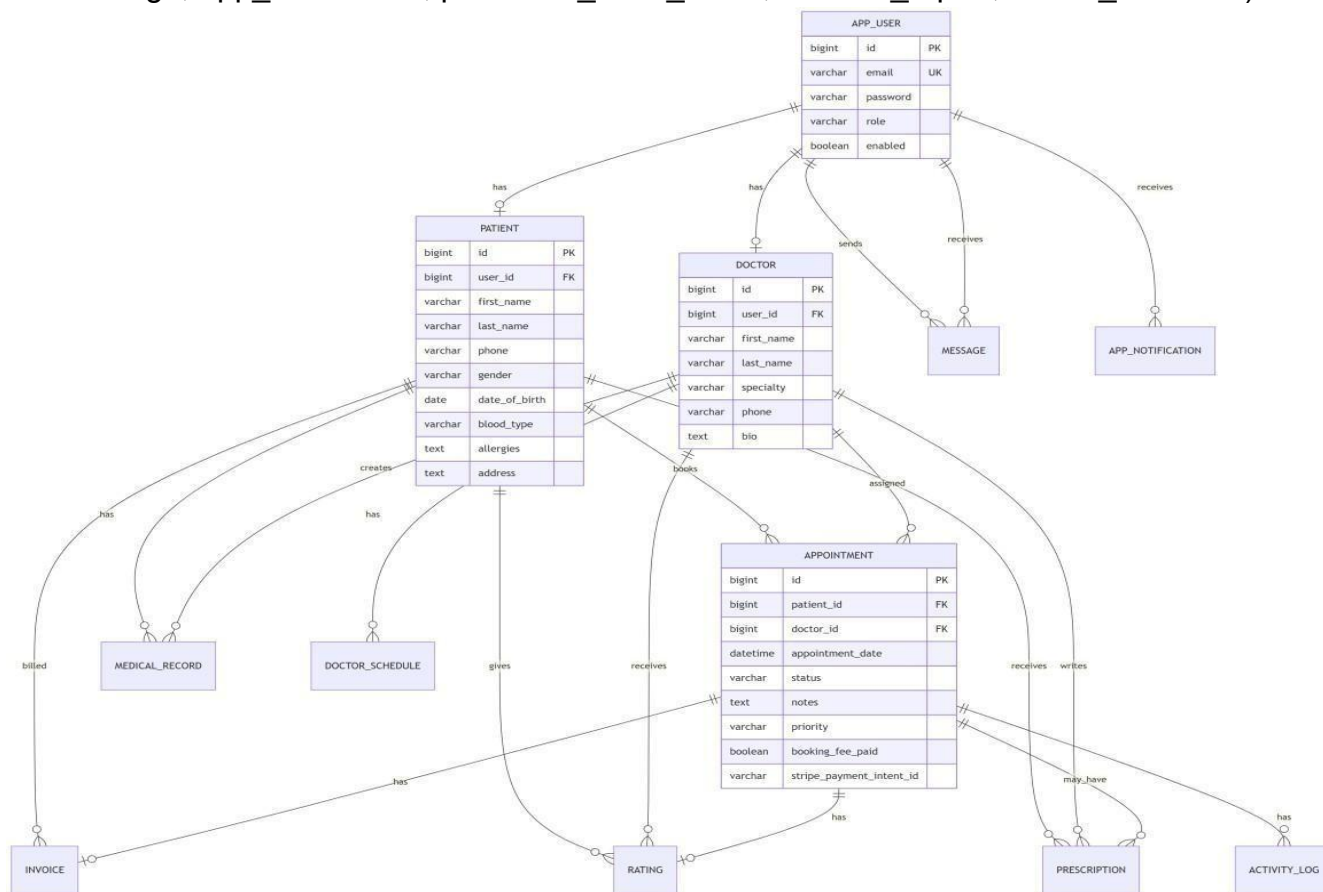


Figura 7. Schema bazei de date (ERD)

Cheii străine sunt configurate cu constrângeri ON DELETE pentru a asigura integritatea referențială. Indexurile sunt definite pe coloanele de tip cheie primară (implicit) și pe emailul utilizatorilor (UNIQUE constraint).

Schema folosește charset utf8mb4 și collation utf8mb4_unicode_ci, oferind suport complet pentru caracterele Unicode, inclusiv diacriticele românești și eventualele emoji din comentariile pacienților.

Pentru schema fizică creată automat de Hibernate, am configurat

spring.jpa.hibernate.ddl-auto=update în mediul de dezvoltare (permite Hibernate să sincronizeze schema cu entitățile JPA) și validate în mediul de producție (doar verifică, nu modifică).

6. Implementare

6.1 Structura pachetelor

Codul Java al aplicației este organizat în pachete după responsabilitatea funcțională, urmând convențiile Java și Spring Boot:

- `com.example.clinic` - pachetul root, conține clasa principală `ClinicApplication` cu metoda `main`.
- `com.example.clinic.config` - clasele de configurare (`SecurityConfig`, `WebMvcConfig`, `OpenApiConfig`, `WebSocketConfig`, `DataLoader`).
- `com.example.clinic.controller` - controller-ele MVC și REST (în total 13 controller-e).
- `com.example.clinic.domain` - entitățile JPA (15 entități).
- `com.example.clinic.dto` - obiectele de transfer (Forms pentru request și Responses pentru REST API).
- `com.example.clinic.exception` - excepțiile custom (6 clase) și `GlobalExceptionHandler`.
- `com.example.clinic.repository` - interfețele Spring Data JPA (15 repository-uri).
- `com.example.clinic.security` - clase auxiliare pentru securitate (`LoginRateLimitInterceptor`).
- `com.example.clinic.service` - serviciile cu logica de business (peste 30 de clase între interfețe și implementări).

Această organizare facilitează navigarea în cod și asigură separarea clară între layere.

6.2 Layerul de prezentare

6.2.1 PatientController

Clasa `PatientController` gestionează toate request-urile HTTP venite de la utilizatorii cu rolul `PATIENT`. Este anotată cu `@RequestMapping("/patient")`, deci toate endpoint-urile sale au prefixul `/patient`. Accesul este restricționat prin configurarea Spring Security.

Dependențele principale, injectate prin constructor, sunt: `PatientService`,

`AppointmentRepository`, `DoctorRepository`, `InvoiceRepository`,
`NotificationService`, `StripeService`, `AiService`, `UserService`, `ActivityLogService`,
`AppNotificationService`, `InvoicePdfService`.

Endpoint-urile principale includ:

- `GET /patient/dashboard` - afișează statistici personale (programări viitoare, notificări necitite, ultimele consultații) și URL-uri rapide către alte funcționalități.

- GET /patient/book - afișează formularul de programare cu lista doctorilor disponibili, filtrabili după specialitate.
- POST /patient/book - creează o programare nouă apelând patientService.bookAppointment(). La succes, redirectează către pagina de plată.
- GET /patient/appointments/{id}/pay - afișează pagina de plată cu detaliile programării.
- GET /patient/history - listează istoricul programărilor cu fișele medicale asociate.
- GET /patient/invoices/{id}/pdf - generează și returnează factura în format PDF.
- POST /patient/appointments/{id}/rate - înregistrează un rating după o consultație finalizată.

Pentru fiecare endpoint care primește un ID de resursă, există o metodă privată verifyPatientOwns() care verifică dacă resursa aparține pacientului autentificat, aruncând AccessDeniedException în caz contrar.

6.2.2 DoctorController

Clasa DoctorController gestionează interfața doctorului. Cele mai importante endpointuri sunt:

- GET /doctor/dashboard - afișează KPI personale (programări azi, total programări, rating mediu, top diagnostice).
- POST /doctor/confirm/{id} - confirmă o programare în starea PENDING. Înainte, verifică prin metoda privată loadAndVerifyAppointment() că programarea aparține doctorului autentificat.
- POST /doctor/cancel/{id} - anulează o programare. Service-ul declanșează automat refund Stripe dacă taxa a fost plătită.
- POST /doctor/complete/{id} - marchează programarea ca finalizată și salvează notele consultației.
- GET /doctor/queue - afișează coada inteligentă cu pacienții programați azi, ordonați după scor de prioritate (calculat de SmartQueueService).
- GET /doctor/calendar - vedere calendaristică interactivă a programărilor.
- POST /doctor/prescriptions - creează o rețetă nouă. Generarea PDF este delegată către PrescriptionService.

6.2.3 AdminController

Controller-ul administratorului expune dashboard-ul cu KPI și statistici, plus accesul la listele de utilizatori, programări, audit log și exportul CSV.

Metoda dashboard() este cea mai complexă, agregând date din mai multe surse:

- Apelează analyticsService.getAdminKpi() pentru numerele principale (total pacienți, doctori, programări, venituri).
- Apelează analyticsService.getMonthlyAppointments(6) pentru datele graficului lunar.
- Apelează analyticsService.getMonthlyRevenue(6) pentru graficul de venituri.

- Apelează `analyticsService.getBusiestHours()` pentru distribuția pe ore.
- Apelează `analyticsService.getTopDoctors(5)` pentru clasament.
- Pune în model listele afișate în modalele care se deschid la click pe KPI cards (`pacientiList`, `doctorsList`, etc.), limitate la primele 50 înregistrări pentru performanță.

Modalele cu liste sunt limitate la 50 înregistrări, suficient pentru o preview rapidă. Pentru lista completă, utilizatorul navighează la pagina dedicată (`/admin/patients`, `/admin/appointments` etc.).

6.2.4 MessagingController

Mesageria internă între pacient și doctor este gestionată de `MessagingController`. Endpointurile sunt duplicate pentru cele două roluri (`/patient/messages` și `/doctor/messages`), dar cu logică similară.

Cea mai sensibilă parte a implementării este deschiderea unui mesaj individual (`GET /{role}/messages/{id}`). Aici, datorită configurării `spring.jpa.open-in-view=false`, sesiunea Hibernate se închide înainte ca Thymeleaf să randeze template-ul. Dacă șablonul accesează `msg.sender.email`, o lazy proxy nu mai poate fi inițializată și se aruncă `LazyInitializationException`.

Soluția adoptată a fost crearea unei metode dedicate în repository:

```
@Query("SELECT m FROM Message m JOIN FETCH m.sender JOIN FETCH m.receiver WHERE m.id = :id")  
Optional<Message> findByIdWithSenderReceiver(@Param("id") Long id);
```

Această metodă încarcă mesajul împreună cu sender-ul și receiver-ul într-un singur query JPQL, eliminând necesitatea unei sesiuni deschise la randare.

6.2.5 PaymentController și StripeController

`PaymentController` inițiază sesiunile Stripe Checkout pentru taxa de rezervare. La click

pe butonul “Plătește” din pagina de plată, controller-ul apelează `stripeService.createCheckoutSession()` cu URL-urile de succes și de eșec, apoi redirecțiază utilizatorul către URL-ul Stripe.

`StripeController` primește webhook-urile Stripe la endpoint-ul `/stripe/webhook`. Acest endpoint este exclus din protecția CSRF în `SecurityConfig`, deoarece nu este apelat de un browser (deci nu are token CSRF), ci de serverele Stripe. În schimb, validarea autenticității requestului se face prin verificarea criptografică a semnăturii:

```
Event event = Webhook.constructEvent(payload, sigHeader, webhookSecret);
```

Dacă semnătura nu se potrivește cu calculul HMAC-SHA256 din webhook secret + payload + timestamp, biblioteca aruncă `SignatureVerificationException` și request-ul este respins cu HTTP 400.

Controller-ul tratează patru tipuri de evenimente Stripe: - checkout.session.completed - marchează taxa ca plătită, schimbă statusul programării din PENDING_PAYMENT în PENDING. - charge.succeeded - finalizează înregistrarea plății cu detaliile sumei și valutei. - charge.refunded - actualizează statusul refund-ului pentru o programare anulată. - charge.failed - loghează eșecul pentru audit, fără modificări în baza de date.

6.2.6 ClinicApiController

Pentru a permite integrări externe și testarea interactivă prin Swagger UI, am implementat un set de endpoint-uri REST publice în ClinicApiController. Acestea returnează JSON și sunt documentate cu adnotări @Operation:

- GET /api/doctors - lista doctorilor sub formă de DoctorResponse.
- GET /api/doctors/{id} - detalii despre un doctor specific.
- GET /api/doctors/specialty/{spec} - filtrare după specialitate.
- GET /api/specialties - lista specialităților distincte.
- GET /api/stats - statistici globale (PlatformStatsResponse).
- GET /api/appointments - programările utilizatorului autentificat (necesită login).
- GET /api/appointments/{id} - detalii despre o programare specifică.

Endpoint-urile sunt accesibile la /swagger-ui/index.html pentru testare interactivă.

6.3 Layerul de business

6.3.1 PatientService

Interfața PatientService (implementată de PatientServiceImpl) conține metodele de business pentru pacienți. Metoda cea mai complexă este bookAppointment():

```
@Override @Transactional public Long bookAppointment(AppointmentForm form, String email) {
    Patient patient = getPatientByEmail(email);
    Doctor doctor = doctorRepository.findById(form.getDoctorId())
        .orElseThrow(() -> new DoctorNotFoundException("id=" + form.getDoctorId()));

    // Validare: data în viitor
    if (form.getAppointmentDate() == null
        || form.getAppointmentDate().isBefore(LocalDate.now())) {
        throw new ClinicException("Data programării trebuie să fie în viitor. ");
    }

    // Validare: în orele de program
    int hour = form.getAppointmentDate().getHour();    if (hour < 8 || hour >= 18) {
        throw new ClinicException("Programările pot fi efectuate doar între 08:00 și 18:00.");
    }

    // Validare: slot disponibil
    long conflicts = appointmentRepository.countConflicts(doctor, form.getAppointmentDate());
    if (conflicts > 0) {
        throw new DuplicateBookingException(doctor.getFullName(), form.getAppointmentDate().toString());
    }

    // Analiza AI a simptomelor
    SymptomAnalysisResult analysis = aiService.analyzeSymptoms(form.getNotes());

    // Salvarea programării
    Appointment appointment = new Appointment();    appointment.setPatient(patient);    appointment.setDoctor(doctor);
    appointment.setAppointmentDate(form.getAppointmentDate());    appointment.setNotes(form.getNotes());
    appointment.setStatus("PENDING_PAYMENT");    appointment.setPriority(analysis.getPriority());
    Appointment saved = appointmentRepository.save(appointment);

    // Audit + notificări + metrice
    auditLogService.log(email, "BOOKING_CREATED", "Appointment #" + saved.getId());
    notificationService.notifyBookingCreated(saved);    metricsService.bookingCreated();    return saved.getId();
}
```

Metoda este anotată cu `@Transactional` pentru a garanta atomicitatea operațiunilor: dacă oricare dintre pași eșuează, toate modificările sunt rolled back.

6.3.2 DoctorService

DoctorService expune operațiunile specifice doctorului. Cele mai importante sunt cele de tranziție de stare a programărilor.

confirmAppointment validează că programarea este în starea PENDING și că taxa de rezervare a fost plătită, apoi schimbă statusul la CONFIRMED și declanșează notificare.

completeAppointment validează că programarea este CONFIRMED, schimbă statusul la COMPLETED, salvează notele consultației și incrementează metrica clinic.bookings.completed.

cancelAppointment anulează programarea indiferent de stare. Dacă taxa fusese plătită, declanșează refund Stripe automat prin apelul stripeService.refund(paymentIntentId).

Tot aici se află metodele de statistici personale: getMonthlyStats() returnează un Map cu cheia "YYYY-MM" pentru graficul lunar, iar getAverageRating() calculează rating-ul mediu pe baza tabelului rating.

6.3.3 AiService

AiServiceImpl implementează două metode publice. recommendSpecialty() returnează numele unei specialități medicale pe baza simptomelor, iar analyzeSymptoms() întoarce un obiect mai complex cu nivelul de urgență.

Strategia de fallback este una din deciziile de design importante. Dacă cheia API DeepSeek nu este configurată sau apelul HTTP eșuează (timeout, eroare de rețea, răspuns invalid), aplicația revine la o euristică bazată pe cuvinte cheie:

```
private SymptomAnalysisResult analyzeByKeywords(String symptoms) {  
    String n = normalize(symptoms);  
    if (containsAny(n, "durere piept", "chest pain", "infarct")) return new  
        SymptomAnalysisResult("CRITICAL", true, "cardiolog", "Dure re în piept");  
    if (containsAny(n, "respira", "lipsa de aer"))  
        return new SymptomAnalysisResult("HIGH", true, "pneumolog", "Dificult ate  
        respiratorie");  
    // ... alte cazuri  
    return new SymptomAnalysisResult("LOW", false, "general", "");  
}
```

Această abordare garantează că aplicația funcționează și fără cheie API, fiind utilă pentru demonstrație și pentru cazurile de fail-over în producție.

6.3.4 StripeService

StripeServiceImpl încapsulează toate apelurile către API-ul Stripe. La inițializare, serviciul verifică dacă cheia secretă este configurată și validă (nu placeholder); dacă nu, aplicația va folosi fallback-ul demonstrativ.

Metoda createCheckoutSession() construiește parametrii sesiunii Stripe (suma, valuta, line items, URL-uri de succes/eșec) și apelează Session.create(params). Pentru a permite retry-uri sigure în caz de eroare de rețea, există o variantă suprapusă care primește un idempotencyKey - Stripe garantează că nu va dubla plata chiar dacă același key este folosit de mai multe ori.

refund(String paymentIntentId) declanșează un refund total. Aplicația păstrează IDul refundului în câmpul stripeRefundId din Appointment pentru audit.

retrieveSession(String sessionId) permite verificarea server-side a stării unei plăți, esențială pentru a nu se baza pe simplul redirect către URL-ul de succes (pe care un atacator l-ar putea apela direct fără să fi plătit).

6.3.5 NotificationService

NotificationServiceImpl trimite notificări prin email (folosind JavaMailSender) și SMS (folosind SDK-ul Twilio). Toate metodele publice sunt anotate cu @Async pentru a nu bloca request-ul HTTP al utilizatorului.

Email-urile folosesc template-uri Thymeleaf separate pentru fiecare tip de notificare: - email/booking-confirmation.html - la crearea programării. - email/paymentsuccess.html - după plata reușită. - email/appointment-confirmed.html - când doctorul confirmă. - email/appointment-completed.html - după finalizarea consultației. - email/appointmentcancelled.html - la anulare. - email/refund-notification.html - după rambursare. - email/appointment-reminder.html - cu 24h înainte (trimis de scheduler). - email/password-reset.html - link de resetare parolă.

Dacă SMTP nu este configurat (variabila MAIL_ENABLED=false), metodele se întorc tăcut fără a arunca excepție. Astfel, aplicația funcționează și în absența configurării completă a notificărilor.

6.3.6 SmartQueueService

SmartQueueService implementează algoritmul de priorizare a cozii de pacienți pentru ziua curentă. Fiecare programare primește un scor compus din:

Scor de bază pe nivelul de prioritate:

- CRITICAL=100, HIGH=75, MEDIUM=50, LOW=25.
- Penalizare pentru orele târzii din zi: hour - 8 (pentru ora 14 → 6 puncte penalizare)

Programările sunt apoi sortate descrescător după scor, plasând pacienții urgenți de dimineață în fața cozii. Doctorul vede astfel o listă inteligent ordonată, care îl ajută să prioritizeze cazurile critice fără a fi nevoie să le caute manual.

Tot aici se află și findBestDoctor(String specialty), care returnează doctorul cu cea mai mică încărcare curentă pentru o specialitate dată - util când pacientul nu are preferință specifică și sistemul îi sugerează un doctor.

6.4 Layerul de acces la date

Repository-urile sunt interfețe simple care extind JpaRepository<Entity, Long>. Spring Data JPA generează automat implementarea la runtime, bazându-se pe convențiile de nume ale metodelor.

De exemplu, în AppointmentRepository:

```
List<Appointment> findByPatient(Patient patient);  
List<Appointment> findByDoctorOrderByAppointmentDateDesc(Doctor doctor); long  
countByStatus(String status);
```

Pentru query-uri complexe, am folosit anotarea @Query cu sintaxă JPQL:

```
@Query("SELECT a FROM Appointment a WHERE a.doctor = :doctor " +  
"AND a.appointmentDate = :date AND a.status != 'CANCELLED'")  
long countConflicts(@Param("doctor") Doctor doctor, @Param("date") LocalDateTime ime date);
```

Această metodă este folosită la validarea slot-urilor disponibile - dacă rezultatul este > 0, slot-ul este deja ocupat.

Pentru rapoarte agregate (cum ar fi distribuția pe ore pentru graficul "Ore aglomerate"), am folosit query-uri JPQL cu GROUP BY:

```
@Query("SELECT HOUR(a.appointmentDate), COUNT(a) FROM Appointment a " +  
"WHERE a.status != 'CANCELLED' " +  
"GROUP BY HOUR(a.appointmentDate) ORDER BY HOUR(a.appointmentDate)")  
List<Object[]> countByHourOfDay();
```

Rezultatul este un List<Object[]> unde fiecare array conține [hour, count], ușor de transformat într-un Map pentru a fi pasat lui Chart.js.

6.5 Securitatea aplicației

Clasa centrală a securității este SecurityConfig, care declară un singur bean: SecurityFilterChain. Configurarea acoperă:

Figura 8. Fragment de cod: configurarea Spring Security

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
        .userDetailsService(userService)
        .csrf(csrf -> csrf.ignoringRequestMatchers("/stripe/webhook", "/ws/**"))
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/", "/register", "/login", "/forgot-password",
                "/reset-password").permitAll()
            .requestMatchers("/css/**", "/js/**", "/images/**").permitAll()
            .requestMatchers("/swagger-ui/**", "/v3/api-docs/**").permitAll()
            .requestMatchers("/api/doctors/**", "/api/stats", "/api/specialties").permitAll()
            .requestMatchers("/patient/**").hasRole("PATIENT")
            .requestMatchers("/doctor/**").hasRole("DOCTOR")
            .requestMatchers("/admin/**").hasRole("ADMIN")
            .anyRequest().authenticated()
        )
        .formLogin(form -> form
            .loginPage("/login")
            .successHandler(new CustomLoginSuccessHandler())
            .permitAll()
        )
        .logout(logout -> logout
            .logoutUrl("/logout")
            .logoutSuccessUrl("/login?logout")
        )
        .headers(headers -> headers
            .frameOptions(frame -> frame.deny())
            .contentTypeOptions(Customizer.withDefaults())
            .referrerPolicy(ref -> ref.policy(SAME_ORIGIN));
    return http.build();
}
```

Rate limiter-ul pentru endpoint-ul /login este implementat în clasa LoginRateLimitInterceptor ca un HandlerInterceptor:

Figura 9. Fragment de cod: rate limiter sliding-window

```
@Override
public boolean preHandle(HttpServletRequest req, HttpServletResponse res, Object handler) {
    if (!"POST".equalsIgnoreCase(req.getMethod())) return true;
    String key = resolveClientIp(req) + ":" + req.getRequestURI();
    long now = System.currentTimeMillis();
    long cutoff = now - WINDOW_MS;
    Deque<Long> window = windowByKey.computeIfAbsent(key, k -> new ArrayDeque<>());
    synchronized (window) {
        while (!window.isEmpty() && window.peekFirst() < cutoff) window.pollFirst();
        if (window.size() >= MAX_ATTEMPTS) {
            res.setStatus(429);
            res.setHeader("Retry-After", String.valueOf((window.peekFirst() + WINDOW_MS - now) / 1000));
            res.sendRedirect("/login?error=rateLimited");
            return false;
        }
        window.addLast(now);
    }
    return true;
}
```

Algoritmul folosește un sliding window de 60 secunde și permite maxim 5 încercări.

Stocarea se face într-un `ConcurrentHashMap<String, Deque<Long>>` cu sincronizare locală pe fiecare deque, pentru a evita contention-ul global.

6.6 Notificările în timp real

Configurarea WebSocket este realizată în `WebSocketConfig`, care implementează interfața `WebSocketMessageBrokerConfigurer`. Endpoint-ul `/ws` este expus cu fallback SockJS, iar brokerul de mesaje folosește prefixele `/topic` (pentru broadcast), `/user` (pentru destinatari specifici) și `/app` (pentru request-uri de la client la server).

`WebSocketNotificationService` folosește un `SimpMessagingTemplate` pentru a trimite mesaje pe topic-uri personalizate per utilizator. De exemplu, la confirmarea unei programări de către doctor, codul apelează:

```
wsNotifications.notifyPatient(appointment, "CONFIRMED", "Programarea ta a fost confirmată de Dr. " + doctorName);
```

Pacientul, dacă are aplicația deschisă în browser, primește instant o notificare toast prin JavaScript.

Integrarea cu Spring Security este facilitată automat: WebSocket-ul moștenește sesiunea HTTP existentă, deci utilizatorul autentificat este deja recunoscut, iar mesajele de pe `/user/queue/notifications` ajung doar la el.

6.7 Integrările externe

1.1.1 Verificarea semnăturii webhook Stripe

Cea mai sensibilă parte a integrării Stripe este endpoint-ul `/stripe/webhook`. Este public (oricine poate trimite POST către el), deci avem nevoie de o metodă de validare alternativă pentru CSRF.

Stripe garantează autenticitatea webhook-urilor prin semnătură criptografică HMAC-

SHA256. Header-ul `Stripe-Signature` conține un timestamp și un hash calculat din (timestamp + payload) semnat cu webhook secret-ul. Biblioteca oficială Stripe Java verifică această semnătură automat:

Figura 10. Fragment de cod: validarea webhook Stripe

```
@PostMapping("/stripe/webhook") public ResponseEntity<String> handleWebhook(@RequestBody String payload,
    @RequestHeader("Stripe-Signature") String sigHeader) {    try {
        Event event = Webhook.constructEvent(payload, sigHeader, webhookSecret);
        switch (event.getType()) {
            case "checkout.session.completed" -> handleSessionCompleted(event);
            case "charge.succeeded" -> handleChargeSucceeded(event);
            case "charge.refunded" -> handleChargeRefunded(event);
            case "charge.failed" -> handleChargeFailed(event);
        }
        return ResponseEntity.ok("");
    } catch (SignatureVerificationException e) {    log.error("Invalid Stripe
signature");    return ResponseEntity.status(400).body("Invalid signature");    }
}
```

1.1.2 Analiza simptomelor cu fallback

AiServiceImpl.analyzeSymptoms() ilustrează patternul de fallback pentru servicii externe:

Figura 11. Fragment de cod: analiza simptomelor cu fallback

```
@Override
public SymptomAnalysisResult analyzeSymptoms(String symptoms) {
    if (symptoms == null || symptoms.isBlank()) {
        return new SymptomAnalysisResult("LOW", false, DEFAULT_SPECIALTY, "");    }
    String n = normalize(symptoms);    // CRITICAL - pune viata in pericol
    if (containsAny(n, "durere piept", "chest pain", "infarct", ...)) {
        return new SymptomAnalysisResult("CRITICAL", true, pickSpecialty(n), buildReason(n));
    }
    // HIGH - urgent    if (containsAny(n, "respira", "lipsa de aer", ...)) {
        return new SymptomAnalysisResult("HIGH", true, pickSpecialty(n), buildReason(n));
    }
    // ... alte cazuri
    return new SymptomAnalysisResult("LOW", false, DEFAULT_SPECIALTY, "");
}
```

Această euristică este compusă din 4 niveluri (CRITICAL/HIGH/MEDIUM/LOW) și acoperă atât termeni medicali în română cât și în engleză, deoarece pacienții pot scrie în oricare dintre limbi.

1.1.3 Conversia numerelor de telefon la format E.164

Twilio acceptă doar numere în format internațional E.164 (de exemplu, +40721234567). Pentru a accepta numere românești în format local (07xx...), implementăm conversia automată:

```
private String toE164(String phone) {  
    String digits = phone.replaceAll("[^0-9+]", "");    if (digits.startsWith("+")) return digits;  
    if (digits.startsWith("0")) return "+4" + digits;    return "+" + digits;  
}
```

7. Testare și instalare

7.1 Testarea automată

Aplicația include o suită de 99 de teste automate, distribuite în 9 clase. Toate testele rulează cu profilul test activat, care folosește o bază de date H2 in-memory pentru a izola complet testele de baza MySQL de dezvoltare.

Tipurile de teste folosite sunt:

Teste unitare - testează o singură clasă cu dependențele mock-uite. Folosesc Mockito pentru a izola unitatea testată. Exemple: PatientServiceImplTest (14 teste), DoctorServiceImplTest (12 teste), AiServiceImplTest (24 teste).

Teste de repository - testează query-urile JPA. Rulează cu un context Spring complet, dar cu baza H2 (pentru viteza). Exemplu: AppointmentRepositoryTest (11 teste).

Teste de securitate - validează că regulile de autorizare RBAC și protecția CSRF funcționează corect. Folosesc MockMvc pentru a simula requesturi HTTP. Exemplu: SecurityAccessTest (17 teste).

Teste de integrare - validează fluxuri complete end-to-end. Folosesc un context Spring complet și baza H2. Exemplu: BookingFlowIntegrationTest (10 teste) care testează fluxul: booking → plată → confirmare → finalizare → rating.

Teste de scheduler - validează că job-urile periodice funcționează corect. Exemple: AppointmentCleanupTaskTest (6 teste), AppointmentReminderTaskTest (4 teste).

Test de smoke - validează că contextul Spring se inițializează fără erori. Exemplu: ClinicApplicationTests (1 test).

Întreaga suită rulează cu comanda `./mvnw test` și durează aproximativ 50 secunde. CI-ul GitHub Actions execută automat suita la fiecare push pe branch-urile main și pe pull request-uri.

7.2 Testarea manuală

Pe lângă testele automate, am parcurs manual următoarele scenarii pentru a verifica experiența utilizatorului final:

- **Scenariu 1: Înregistrare și autentificare ca pacient**
 1. Accesez <http://localhost:8082/register>.
 2. Completez formularul: prenume "Test", nume "Pacient", email "test@example.com", parolă "Test1234", telefon "0721000000".
 3. Verific redirectarea la /login?registered.
 4. Mă autentific cu credentialele introduse.
 5. Verific redirectarea la /patient/dashboard.
- **Scenariu 2: Crearea unei programări**
 1. Din dashboard, click pe "Programare nouă".
 2. Selectez doctor "Dr. Alexandru Popescu" (Medicină Generală).
 3. Aleg data de mâine, ora 10:00.
 4. Descriu simptomele: "Dureri de cap frecvente, oboseală cronică".
 5. Submit. Verific redirectarea la pagina de plată cu prioritatea calculată (MEDIUM).
- **Scenariu 3: Plata taxei de rezervare (mod demonstrativ)**
 1. Pe pagina de plată, click "Plătește".
 2. Sistemul folosește fallback-ul demonstrativ (Stripe nu e configurat).
 3. Statusul programării se schimbă la PENDING.
 4. Verific apariția notificării în UI.
- **Scenariu 4: Confirmarea de către doctor**
 1. Deconectare și autentificare ca doctor1@clinic.com.
 2. Acces la /doctor/dashboard, vizualizare programare nou creată.
 3. Click pe "Confirmă" pentru programarea respectivă.
 4. Verific schimbarea statusului la CONFIRMED și notificarea trimisă (vizibilă în log).
- **Scenariu 5: Finalizarea consultației**
 1. Doctorul navighează la programarea CONFIRMED.
 2. Click "Finalizează consultația".
 3. Introduce note: "Consultație efectuată. Pacient stabil. Control în 3 luni."
 4. Salvare. Statusul devine COMPLETED.
- **Scenariu 6: Vizualizarea KPI ca administrator**
 1. Deconectare și autentificare ca admin@clinic.com.
 2. Acces la /admin/dashboard.
 3. Verific afișarea corectă a KPI-urilor: total pacienți, doctori, programări.
 4. Verific funcționarea graficelor Chart.js.
 5. Click pe cardul "Programări în așteptare" pentru a deschide modal-ul cu lista.

7.3 Instalarea locală

Pentru instalarea locală a aplicației, sunt necesare următoarele cerințe preliminare:

- **Java Development Kit 21** sau mai recent
- **MySQL 8** (recomandat XAMPP pentru dezvoltare, oferă instalare rapidă)
- **Git** (opțional, pentru clonarea repository-ului) Pașii de instalare sunt:

Pas 1: Clonarea repository-ului

```
git clone https://github.com/dawood-ahmed/clinic-licenta.git cd clinic-licenta
```

Pas 2: Crearea bazei de date

```
mysql -u root -e "CREATE DATABASE clinic CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;"
```

Pas 3: Configurarea credențialelor

Dacă credențialele MySQL diferă de cele implicite (root, fără parolă), se modifică în fișierul src/main/resources/application.properties:

```
spring.datasource.username=root  
spring.datasource.password=your_password_here
```

Pas 4: Pornirea aplicației

```
./mvnw spring-boot:run
```

Pas 5: Accesarea aplicației

Aplicația este disponibilă la <http://localhost:8082>. La prima pornire, DataLoader încarcă automat un set de date de test cuprinzând 12 doctori, 64 pacienți, 279 programări, plus fișe medicale, rețete, facturi și rating-uri.

7.4 Instalarea cu Docker

Pentru o instalare mai izolată, se poate folosi Docker

Compose: `cp .env.example .env`

```
# Se editează .env și se setează parolele MySQL  
docker-compose up --build
```

Acest comanda pornește trei containere: - **clinic-app** pe portul 8082 - aplicația Spring Boot - **clinic-db** pe portul 3307 - MySQL 8 - **clinic-phpmysqladmin** pe portul 8081 - interfață web pentru administrarea DB

7.5 Deploy în Google Cloud Run

Pentru deploy în producție, aplicația este pregătită pentru Google Cloud Run + Cloud SQL (Figura 15).

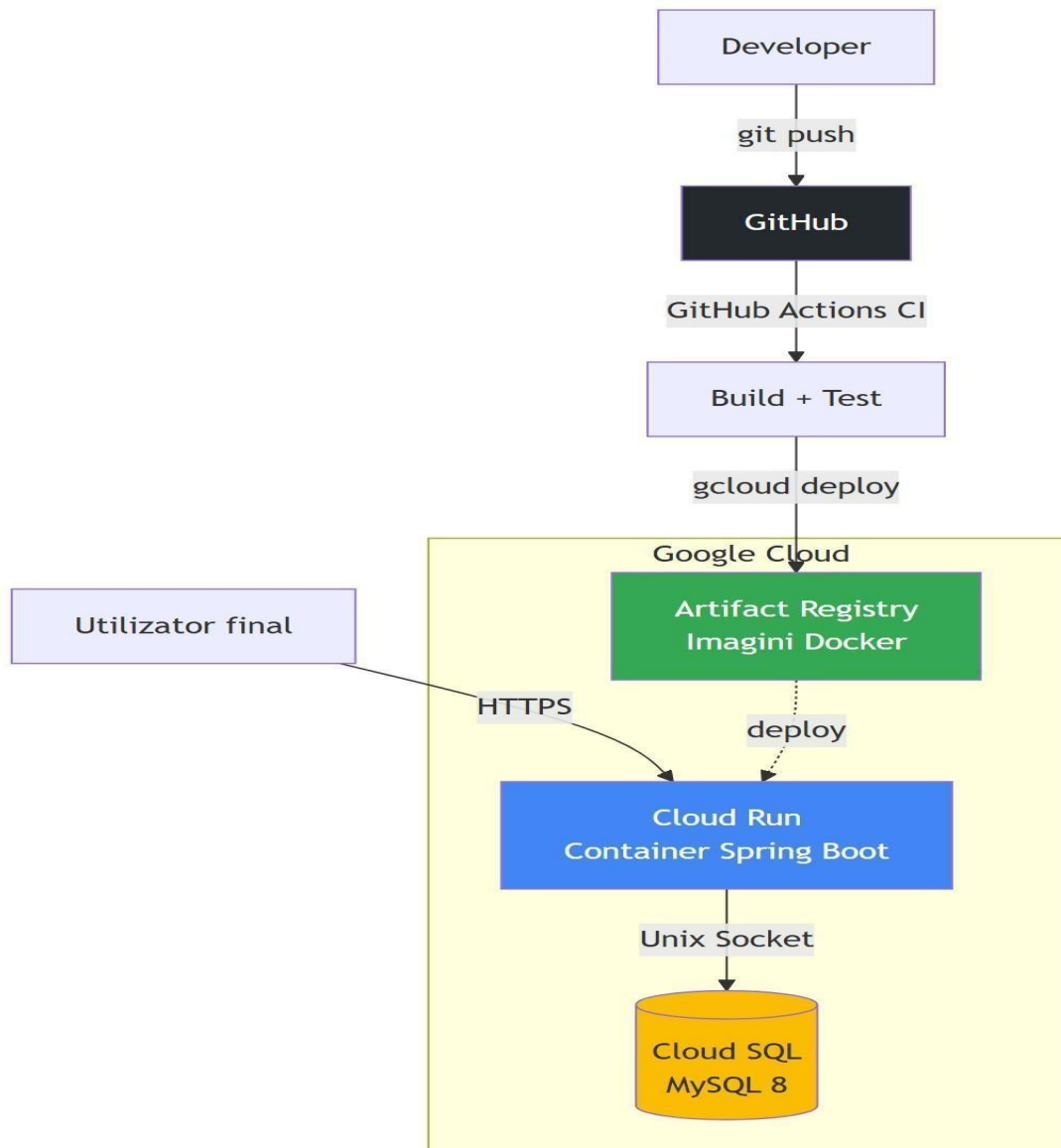


Figura 8. Arhitectura de deploy in Google Cloud

Run Pașii principali:

1. **Creare proiect Google Cloud** și activare API-uri necesare (Cloud Run, Cloud SQL, Artifact Registry).
2. **Creare instanță Cloud SQL** (MySQL 8) cu nivelul db-f1-micro.
3. **Build și push imagine Docker** către Artifact Registry.
4. **Deploy pe Cloud Run** cu configurarea variabilelor de mediu (SPRING_PROFILES_ACTIVE=cloud, credențiale DB, cheile API).

Costurile estimate sunt aproximativ 7-10 USD pe lună după consumarea creditului gratuit de 300 USD pe care Google îl oferă conturilor noi (valabil 90 zile).

Detaliile complete sunt documentate în fișierul DEPLOY-GCLOUD.md din repository.

8. Concluzii și implementări ulterioare

La sfârșitul acestei lucrări pot spune că obiectivele pe care mi le-am propus la început au fost îndeplinite. Am construit o aplicație web funcțională, cu toate cele trei tipuri de utilizatori, cu plăți integrate, cu analiză AI, cu notificări în timp real și cu o suită serioasă de testare automată. Aplicația rulează atât local, cât și în Docker, cât și în cloud pe Google Cloud Run.

Ce m-a surprins cel mai mult pe parcurs a fost cât de mult timp se duce pe părți pe care la început le credeam triviale. De exemplu, integrarea Stripe — citisem documentația și mi se părea clară, dar verificarea semnăturii webhook-ului mi-a luat câteva zile bune până am înțeles exact ce e de făcut. La fel cu LazyInitializationException — la prima vedere părea o eroare simplă, dar până am înțeles relația dintre open-in-view=false, sesiunea Hibernate și momentul randării template-ului, mi-a luat ceva timp. Am rezolvat-o adăugând JOIN FETCH explicit în query-urile JPQL, dar lecția mai importantă a fost că antipatternurile evitate pot introduce probleme noi care trebuie tratate cu grijă.

O decizie de design de care sunt mulțumit a fost să fac fallback-uri peste tot. Pentru Stripe — dacă nu am cheie API, plata se simulează local și aplicația merge mai departe. Pentru DeepSeek — dacă API-ul nu răspunde sau lipsește cheia, codul cade pe o euristică pe cuvinte cheie pe care am scris-o eu, suficient de bună pentru a clasifica cazurile evidente (durere în piept → CRITICAL, control de rutină → LOW). Pentru emailuri și SMSuri — dacă nu e configurat SMTP-ul sau Twilio, metodele se întorc tăcut. Asta face aplicația ușor de demonstrat fără să trebuiască să dau cuiva chei reale, iar în producție garantează că nu cade tot doar pentru că DeepSeek e momentan indisponibil.

Pe partea de securitate am încercat să acopăr cât mai multe vectoare relevante pentru o aplicație web obișnuită. Parolele sunt BCrypt cu cost 10. Sesiunile au cookie-uri HttpOnly cu SameSite=Lax și expiră după 30 minute. CSRF e activ pe absolut toate formularele POST. La login există un rate limiter pe care l-am scris în Java pur, fără biblioteci externe, care folosește sliding window și se sincronizează local pe fiecare deque ca să nu am contention global. Headerele HTTP standard sunt setate la fiecare răspuns (X-Frame-Options, XContent-Type-Options, Referrer-Policy).

Ce regret puțin e că nu am avut timp să integrez Flyway pentru migrările de schemă — momentan folosesc ddl-auto=update care e ok pentru dezvoltare, dar într-un sistem real ar fi nevoie de migrații versionate. Am pus-o pe lista de îmbunătățiri viitoare.

Direcții de dezvoltare viitoare

Sunt câteva idei pe care, dacă aș continua proiectul după licență, m-aș apuca să le adaug:

1. **Aplicație mobilă nativă** - dezvoltarea unei aplicații în React Native sau Flutter care să consume REST API-ul existent, oferind o experiență mai naturală pe dispozitive mobile. Notificările push native (prin Firebase Cloud Messaging) ar înlocui parțial WebSocket-ul.
2. **Telemedicină prin video-consultații** - integrarea unui serviciu WebRTC (Twilio Video, Daily.co sau Jitsi self-hosted) pentru consultații la distanță, esențial pentru zonele rurale sau pentru pacienții cu mobilitate redusă.
3. **Multi-tenancy** - extinderea arhitecturii pentru a permite găzduirea mai multor clinici independente pe aceeași instanță, cu izolare strictă la nivel de date prin discriminator de tenant.
4. **Chatbot AI conversațional** - un asistent virtual care răspunde la întrebări frecvente ale pacienților, integrat eventual cu informațiile din fișa medicală personală pentru sfaturi personalizate.
5. **Integrare cu sisteme naționale de sănătate** - conectarea cu sistemul CNAS pentru emiterea de bilete de trimitere și rețete electronice oficiale, plus integrarea cu Dosarul Electronic de Sănătate al pacientului.
6. **Predicție analitică pentru optimizare** - folosirea istoricului consultațiilor pentru a prezice cererea pe specialități și a optimiza programările doctorilor, eventual cu rezervare automată de sloturi suplimentare în zilele cu cerere mare.
7. **Backup automat și disaster recovery** - configurarea unui job zilnic care exportă baza de date și o încarcă într-un bucket Google Cloud Storage (sau echivalent), plus un mecanism de restore documentat.
8. **Internaționalizare (i18n)** - mutarea string-urilor UI într-un fișier messages_ro.properties și adăugarea unui messages_en.properties pentru pacienții non-vorbitori de română.
9. **Audit log avansat cu căutare full-text** - integrarea cu un sistem de logging extern (Elasticsearch + Kibana) pentru audit log mai sofisticat, cu căutări complexe și alertare la anomalii.
10. **Plăți cu carduri locale și instrumente alternative** - integrarea cu procesatori locali (Netopia, Mobilpay) pentru a accepta carduri emise de bănci românești care nu funcționează cu Stripe.

9. Bibliografie

- [1] L. Bass, P. Clements și R. Kazman, *Software Architecture in Practice*, ed. a 4-a. Boston, MA: Addison-Wesley Professional, 2021.
- [2] R. C. Martin, *Clean Code: A Handbook of Agile Software Craftsmanship*. Upper Saddle River, NJ: Prentice Hall, 2008.
- [3] J. Bloch, *Effective Java*, ed. a 3-a. Boston, MA: Addison-Wesley Professional, 2018.
- [4] A. Wiggins, "The Twelve-Factor App," 2012. [Online]. Disponibil: <https://12factor.net/>
- [5] E. Gamma, R. Helm, R. Johnson și J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley Professional, 1994.
- [6] M. Fowler, *Patterns of Enterprise Application Architecture*. Boston, MA: AddisonWesley Professional, 2002.
- [7] C. Walls, *Spring in Action*, ed. a 6-a. Shelter Island, NY: Manning Publications, 2022.
- [8] V. Khorikov, *Unit Testing Principles, Practices, and Patterns*. Shelter Island, NY: Manning Publications, 2020.
- [9] Spring Boot Team, *Spring Boot Reference Documentation, version 4.0.5*, VMware Tanzu, 2025. [Online]. Disponibil: <https://docs.spring.io/springboot/docs/current/reference/html/>
- [10] Spring Security Team, *Spring Security Reference, version 7.0*, VMware Tanzu, 2025. [Online]. Disponibil: <https://docs.spring.io/spring-security/reference/>
- [11] Spring Data Team, *Spring Data JPA Reference Documentation*, VMware Tanzu, 2025. [Online]. Disponibil: <https://docs.spring.io/spring-data/jpa/reference/>
- [12] Hibernate ORM Team, *Hibernate ORM User Guide*, Red Hat, 2025. [Online]. Disponibil: <https://hibernate.org/orm/documentation/>
- [13] Oracle Corporation, *Java SE 21 Documentation*, 2023. [Online]. Disponibil: <https://docs.oracle.com/en/java/javase/21/>
- [14] MySQL Documentation Team, *MySQL 8.0 Reference Manual*, Oracle, 2025. [Online]. Disponibil: <https://dev.mysql.com/doc/refman/8.0/en/>
- [15] Thymeleaf Team, *Thymeleaf 3 Documentation*, 2025. [Online]. Disponibil: <https://www.thymeleaf.org/documentation.html>
- [16] Bootstrap Team, *Bootstrap 5.3 Documentation*, 2025. [Online]. Disponibil: <https://getbootstrap.com/docs/5.3/>

- [17] Stripe Inc., *Stripe API Reference*, 2025. [Online]. Disponibil: <https://docs.stripe.com/api>
- [18] DeepSeek AI, *DeepSeek API Documentation*, 2024. [Online]. Disponibil: <https://apidocs.deepseek.com/>
- [19] Twilio Inc., *Twilio SMS API Documentation*, 2025. [Online]. Disponibil: <https://www.twilio.com/docs/sms>
- [20] Docker Inc., *Docker Documentation*, 2025. [Online]. Disponibil: <https://docs.docker.com/>
- [21] Google Cloud, *Cloud Run Documentation*, 2025. [Online]. Disponibil: <https://cloud.google.com/run/docs>
- [22] OpenAPI Initiative, *OpenAPI Specification 3.1.0*, 2024. [Online]. Disponibil: <https://spec.openapis.org/oas/v3.1.0>
- [23] R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," teză de doctorat, Univ. of California, Irvine, CA, 2000.
- [24] N. Provos și D. Mazieres, "A Future-Adaptable Password Scheme," în *Proc. USENIX Annual Technical Conf.*, Monterey, CA, 1999, pp. 81-92.
- [25] M. Belshe, R. Peon și M. Thomson, "Hypertext Transfer Protocol Version 2 (HTTP/2)," RFC 7540, Internet Engineering Task Force, 2015. [Online]. Disponibil: <https://datatracker.ietf.org/doc/html/rfc7540>
- [26] OWASP Foundation, *OWASP Top 10 - 2021: The Ten Most Critical Web Application Security Risks*, 2021. [Online]. Disponibil: <https://owasp.org/Top10/>
- [27] Eurostat, "Digitalisation in Europe - 2023 Statistics on the Digital Economy and Society," European Commission, 2023. [Online]. Disponibil: <https://ec.europa.eu/eurostat/>
- [28] A. Barth, "The Web Origin Concept," RFC 6454, Internet Engineering Task Force, 2011. [Online]. Disponibil: <https://datatracker.ietf.org/doc/html/rfc6454>
- [29] Baeldung, *Baeldung - Java, Spring and Web Development Tutorials*, 2025. [Online]. Disponibil: <https://www.baeldung.com/>
- [30] Mozilla Developer Network, *MDN Web Docs*, 2025. [Online]. Disponibil: <https://developer.mozilla.org/>
- [31] M. Vlăduțiu, "Tehnologii Web," curs anul III, Facultatea de Automatică și Calculatoare, Universitatea Politehnica Timișoara, 2024-2025.
- [32] I. Jian, "Baze de Date," curs anul II, Facultatea de Automatică și Calculatoare, Universitatea Politehnica Timișoara, 2023-2024.

- [33] D. Drăniduș, “Ingineria Sistemelor de Programe,” curs anul III, Facultatea de Automatică și Calculatoare, Universitatea Politehnica Timișoara, 2024-2025.
- [34] M. Crețu, “Programare Orientată pe Obiecte,” curs anul II, Facultatea de Automatică și Calculatoare, Universitatea Politehnica Timișoara, 2023-2024.
- [35] Stack Overflow, *Programming Q&A Community*, 2025. [Online]. Disponibil: <https://stackoverflow.com/>